fill-ins, initially zero elements that must become nonzero during the solution process, and for which storage must be reserved.

Direct methods for solving sparse equations, then, depend crucially on the precise pattern of sparsity of the matrix. Patterns that occur frequently, or that are useful as way stations in the reduction of more general forms, already have special names and special methods of solution. We do not have space here for any detailed review of these. References listed at the end of this section will furnish you with an "in" to the specialized literature, and the following list of buzz words (and Figure 2.7.1) will at least let you hold your own at cocktail parties:

- tridiagonal
- band-diagonal (or banded) with bandwidth M
- band triangular
- block diagonal
- block tridiagonal
- block triangular
- cyclic banded
- singly (or doubly) bordered block diagonal
- singly (or doubly) bordered block triangular
- singly (or doubly) bordered band-diagonal
- singly (or doubly) bordered band triangular
- other (!)

You should also be aware of some of the special sparse forms that occur in the solution of partial differential equations in two or more dimensions. See Chapter 20.

If your particular pattern of sparsity is not a simple one, then you may wish to try an *analyze/factorize/operate* package, which automates the procedure of figuring out how fill-ins are to be minimized. The *analyze* stage is done once only for each pattern of sparsity. The *factorize* stage is done once for each particular matrix that fits the pattern. The *operate* stage is performed once for each right-hand side to be used with the particular matrix. Consult [2,3] for references on this. The NAG library [4] has an analyze/factorize/operate capability. A substantial collection of routines for sparse matrix calculation is also available from IMSL [5] as the *Yale Sparse Matrix Package* [6].

You should be aware that the special order of interchanges and eliminations, prescribed by a sparse matrix method so as to minimize fill-ins and arithmetic operations, generally acts to decrease the method's numerical stability as compared to, e.g., regular LU decomposition with pivoting. Scaling your problem so as to make its nonzero matrix elements have comparable magnitudes (if you can do it) will sometimes ameliorate this problem.

In the remainder of this section, we present some concepts that are applicable to some general classes of sparse matrices, and which do not necessarily depend on details of the pattern of sparsity.

2.7.1 Sherman-Morrison Formula

Suppose that you have already obtained, by herculean effort, the inverse matrix $\mathbf{A}^{-1}$ of a square matrix $\mathbf{A}$. Now you want to make a "small" change in $\mathbf{A}$, for example change one element a_{ij}, or a few elements, or one row, or one column. Is there any

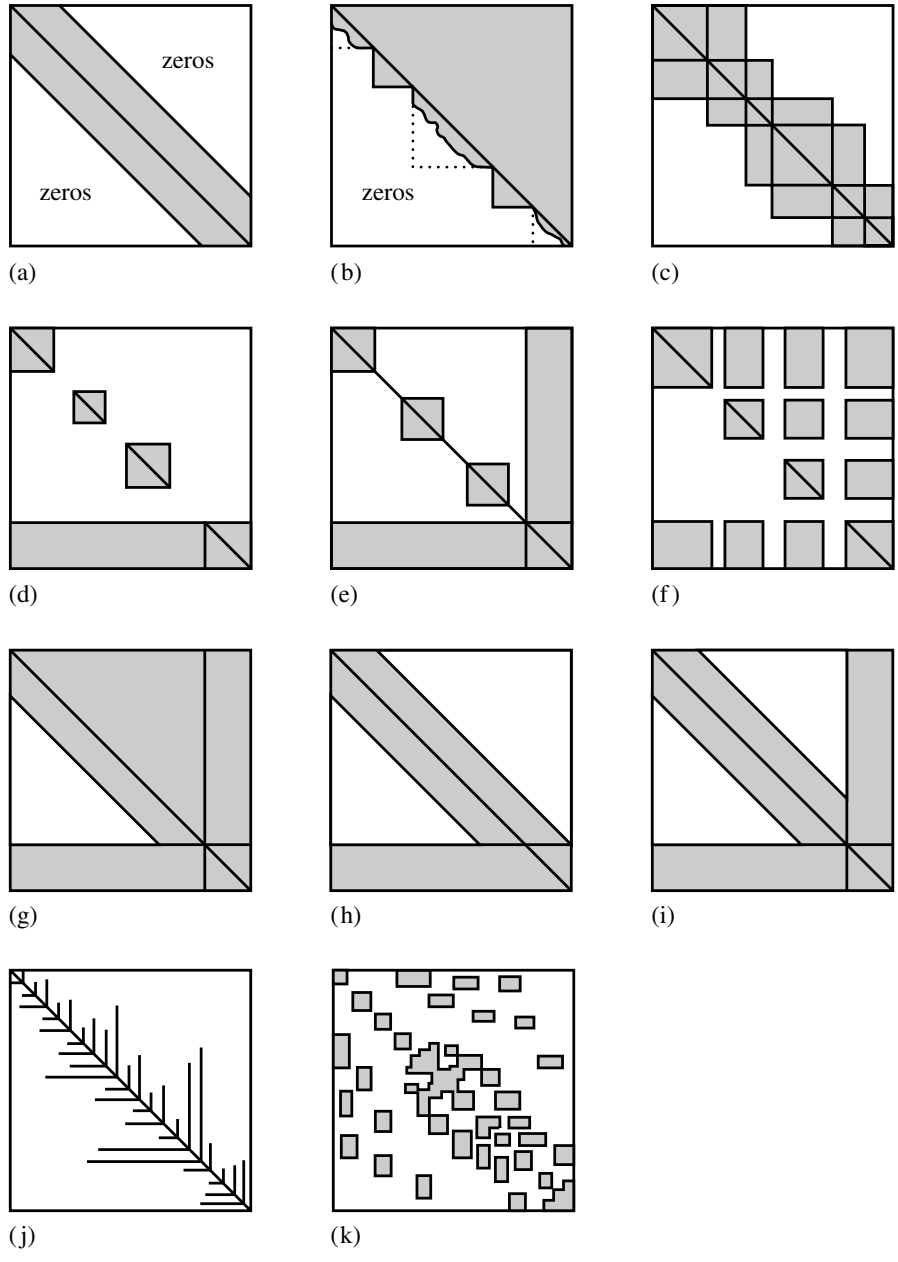


Figure 2.7.1. Some standard forms for sparse matrices. (a) Band-diagonal; (b) block triangular; (c) block tridiagonal; (d) singly bordered block diagonal; (e) doubly bordered block diagonal; (f) singly bordered block triangular; (g) bordered band-triangular; (h) and (i) singly and doubly bordered band-diagonal; (j) and (k) other! (after Tewarson) [1].

way of calculating the corresponding change in $\mathbf{A}^{-1}$ without repeating your difficult labors? Yes, if your change is of the form

$$\mathbf{A} \rightarrow (\mathbf{A} + \mathbf{u} \otimes \mathbf{v}) \tag{2.7.1}$$

for some vectors $\mathbf{u}$ and $\mathbf{v}$. If $\mathbf{u}$ is a unit vector $\mathbf{e}_i$, then (2.7.1) adds the components of $\mathbf{v}$ to the ith row. (Recall that $\mathbf{u} \otimes \mathbf{v}$ is a matrix whose i,jth element is the product of the ith component of $\mathbf{u}$ and the jth component of $\mathbf{v}$.) If $\mathbf{v}$ is a unit vector $\mathbf{e}_j$, then (2.7.1) adds the components of $\mathbf{u}$ to the jth column. If both $\mathbf{u}$ and $\mathbf{v}$ are proportional to unit vectors $\mathbf{e}_i$ and $\mathbf{e}_j$, respectively, then a term is added only to the element a_{ij}.

The *Sherman-Morrison* formula gives the inverse $(\mathbf{A} + \mathbf{u} \otimes \mathbf{v})^{-1}$ and is derived briefly as follows:

$$\begin{aligned}(\mathbf{A} + \mathbf{u} \otimes \mathbf{v})^{-1} &= (\mathbf{1} + \mathbf{A}^{-1} \cdot \mathbf{u} \otimes \mathbf{v})^{-1} \cdot \mathbf{A}^{-1} \\ &= (\mathbf{1} - \mathbf{A}^{-1} \cdot \mathbf{u} \otimes \mathbf{v} + \mathbf{A}^{-1} \cdot \mathbf{u} \otimes \mathbf{v} \cdot \mathbf{A}^{-1} \cdot \mathbf{u} \otimes \mathbf{v} - \ldots) \cdot \mathbf{A}^{-1} \\ &= \mathbf{A}^{-1} - \mathbf{A}^{-1} \cdot \mathbf{u} \otimes \mathbf{v} \cdot \mathbf{A}^{-1} \, (1 - \lambda + \lambda^2 - \ldots) \\ &= \mathbf{A}^{-1} - \frac{(\mathbf{A}^{-1} \cdot \mathbf{u}) \otimes (\mathbf{v} \cdot \mathbf{A}^{-1})}{1 + \lambda} \end{aligned} \tag{2.7.2}$$

where

$$\lambda \equiv \mathbf{v} \cdot \mathbf{A}^{-1} \cdot \mathbf{u} \tag{2.7.3}$$

The second line of (2.7.2) is a formal power series expansion. In the third line, the associativity of outer and inner products is used to factor out the scalars λ.

The use of (2.7.2) is this: Given $\mathbf{A}^{-1}$ and the vectors $\mathbf{u}$ and $\mathbf{v}$, we need only perform two matrix multiplications and a vector dot product,

$$\mathbf{z} \equiv \mathbf{A}^{-1} \cdot \mathbf{u} \qquad \mathbf{w} \equiv (\mathbf{A}^{-1})^T \cdot \mathbf{v} \qquad \lambda = \mathbf{v} \cdot \mathbf{z} \tag{2.7.4}$$

to get the desired change in the inverse

$$\mathbf{A}^{-1} \quad \rightarrow \quad \mathbf{A}^{-1} - \frac{\mathbf{z} \otimes \mathbf{w}}{1 + \lambda} \tag{2.7.5}$$

The whole procedure requires only $3N^2$ multiplies and a like number of adds (an even smaller number if $\mathbf{u}$ or $\mathbf{v}$ is a unit vector).

The Sherman-Morrison formula can be directly applied to a class of sparse problems. If you already have a fast way of calculating the inverse of $\mathbf{A}$ (e.g., a tridiagonal matrix or some other standard sparse form), then (2.7.4) – (2.7.5) allow you to build up to your related but more complicated form, adding for example a row or column at a time. Notice that you can apply the Sherman-Morrison formula more than once successively, using at each stage the most recent update of $\mathbf{A}^{-1}$ (equation 2.7.5). Of course, if you have to modify *every* row, then you are back to an N^3 method. The constant in front of the N^3 is only a few times worse than the better direct methods, but you have deprived yourself of the stabilizing advantages of pivoting — so be careful.

For some other sparse problems, the Sherman-Morrison formula cannot be directly applied for the simple reason that storage of the whole inverse matrix $\mathbf{A}^{-1}$ is not feasible. If you want to add only a single correction of the form $\mathbf{u} \otimes \mathbf{v}$ and solve the linear system

$$(\mathbf{A} + \mathbf{u} \otimes \mathbf{v}) \cdot \mathbf{x} = \mathbf{b} \tag{2.7.6}$$

then you proceed as follows. Using the fast method that is presumed available for the matrix $\mathbf{A}$, solve the two auxiliary problems

$$\mathbf{A} \cdot \mathbf{y} = \mathbf{b} \qquad \mathbf{A} \cdot \mathbf{z} = \mathbf{u} \tag{2.7.7}$$

for the vectors $\mathbf{y}$ and $\mathbf{z}$. In terms of these,

$$\mathbf{x} = \mathbf{y} - \left[\frac{\mathbf{v}\cdot\mathbf{y}}{1+(\mathbf{v}\cdot\mathbf{z})}\right]\mathbf{z} \tag{2.7.8}$$

as we see by multiplying (2.7.2) on the right by $\mathbf{b}$.

2.7.2 Cyclic Tridiagonal Systems

So-called *cyclic tridiagonal systems* occur quite frequently and are a good example of how to use the Sherman-Morrison formula in the manner just described. The equations have the form

$$\begin{bmatrix} b_0 & c_0 & 0 & \cdots & & & \beta \\ a_1 & b_1 & c_1 & \cdots & & & \\ & & & \cdots & & & \\ & & & \cdots & a_{N-2} & b_{N-2} & c_{N-2} \\ \alpha & & & \cdots & 0 & a_{N-1} & b_{N-1} \end{bmatrix} \cdot \begin{bmatrix} x_0 \\ x_1 \\ \cdots \\ x_{N-2} \\ x_{N-1} \end{bmatrix} = \begin{bmatrix} r_0 \\ r_1 \\ \cdots \\ r_{N-2} \\ r_{N-1} \end{bmatrix} \tag{2.7.9}$$

This is a tridiagonal system, except for the matrix elements α and β in the corners. Forms like this are typically generated by finite differencing differential equations with periodic boundary conditions (§20.4).

We use the Sherman-Morrison formula, treating the system as tridiagonal plus a correction. In the notation of equation (2.7.6), define vectors $\mathbf{u}$ and $\mathbf{v}$ to be

$$\mathbf{u} = \begin{bmatrix} \gamma \\ 0 \\ \vdots \\ 0 \\ \alpha \end{bmatrix} \qquad \mathbf{v} = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \\ \beta/\gamma \end{bmatrix} \tag{2.7.10}$$

Here γ is arbitrary for the moment. Then the matrix $\mathbf{A}$ is the tridiagonal part of the matrix in (2.7.9), with two terms modified:

$$b_0' = b_0 - \gamma, \qquad b_{N-1}' = b_{N-1} - \alpha\beta/\gamma \tag{2.7.11}$$

We now solve equations (2.7.7) with the standard tridiagonal algorithm and then get the solution from equation (2.7.8).

The routine `cyclic` below implements this algorithm. We choose the arbitrary parameter $\gamma = -b_0$ to avoid loss of precision by subtraction in the first of equations (2.7.11). In the unlikely event that this causes loss of precision in the second of these equations, you can make a different choice.

tridag.h

```
void cyclic(VecDoub_I &a, VecDoub_I &b, VecDoub_I &c, const Doub alpha,
    const Doub beta, VecDoub_I &r, VecDoub_O &x)
```

Solves for a vector `x[0..n-1]` the "cyclic" set of linear equations given by equation (2.7.9). `a`, `b`, `c`, and `r` are input vectors, all dimensioned as `[0..n-1]`, while `alpha` and `beta` are the corner entries in the matrix. The input is not modified.

```
{
    Int i,n=a.size();
    Doub fact,gamma;
    if (n <= 2) throw("n too small in cyclic");
```

```
    VecDoub bb(n),u(n),z(n);
    gamma = -b[0];                          Avoid subtraction error in forming bb[0].
    bb[0]=b[0]-gamma;                       Set up the diagonal of the modified tridi-
    bb[n-1]=b[n-1]-alpha*beta/gamma;            agonal system.
    for (i=1;i<n-1;i++) bb[i]=b[i];
    tridag(a,bb,c,r,x);                     Solve A · x = r.
    u[0]=gamma;                             Set up the vector u.
    u[n-1]=alpha;
    for (i=1;i<n-1;i++) u[i]=0.0;
    tridag(a,bb,c,u,z);                     Solve A · z = u.
    fact=(x[0]+beta*x[n-1]/gamma)/          Form v · x/(1 + v · z).
        (1.0+z[0]+beta*z[n-1]/gamma);
    for (i=0;i<n;i++) x[i] -= fact*z[i];    Now get the solution vector x.
}
```

2.7.3 Woodbury Formula

If you want to add more than a single correction term, then you cannot use (2.7.8) repeatedly, since without storing a new $\mathbf{A}^{-1}$ you will not be able to solve the auxiliary problems (2.7.7) efficiently after the first step. Instead, you need the *Woodbury formula*, which is the block-matrix version of the Sherman-Morrison formula,

$$(\mathbf{A} + \mathbf{U} \cdot \mathbf{V}^T)^{-1} = \mathbf{A}^{-1} - \left[\mathbf{A}^{-1} \cdot \mathbf{U} \cdot (\mathbf{1} + \mathbf{V}^T \cdot \mathbf{A}^{-1} \cdot \mathbf{U})^{-1} \cdot \mathbf{V}^T \cdot \mathbf{A}^{-1}\right] \tag{2.7.12}$$

Here A is, as usual, an $N \times N$ matrix, while $\mathbf{U}$ and $\mathbf{V}$ are $N \times P$ matrices with $P < N$ and usually $P \ll N$. The inner piece of the correction term may become clearer if written as the tableau,

$$\begin{bmatrix} & \\ \mathbf{U} & \\ & \end{bmatrix} \cdot \left[\mathbf{1} + \mathbf{V}^T \cdot \mathbf{A}^{-1} \cdot \mathbf{U}\right]^{-1} \cdot \begin{bmatrix} & \\ \mathbf{V}^T & \\ & \end{bmatrix} \tag{2.7.13}$$

where you can see that the matrix whose inverse is needed is only $P \times P$ rather than $N \times N$.

The relation between the Woodbury formula and successive applications of the Sherman-Morrison formula is now clarified by noting that, if $\mathbf{U}$ is the matrix formed by columns out of the P vectors $\mathbf{u}_0, \ldots, \mathbf{u}_{P-1}$, and $\mathbf{V}$ is the matrix formed by columns out of the P vectors $\mathbf{v}_0, \ldots, \mathbf{v}_{P-1}$,

$$\mathbf{U} \equiv \begin{bmatrix} & & \\ \mathbf{u}_0 & \cdots & \mathbf{u}_{P-1} \\ & & \end{bmatrix} \qquad \mathbf{V} \equiv \begin{bmatrix} & & \\ \mathbf{v}_0 & \cdots & \mathbf{v}_{P-1} \\ & & \end{bmatrix} \tag{2.7.14}$$

then two ways of expressing the same correction to $\mathbf{A}$ are

$$\left(\mathbf{A} + \sum_{k=0}^{P-1} \mathbf{u}_k \otimes \mathbf{v}_k\right) = (\mathbf{A} + \mathbf{U} \cdot \mathbf{V}^T) \tag{2.7.15}$$

(Note that the subscripts on $\mathbf{u}$ and $\mathbf{v}$ do *not* denote components, but rather distinguish the different column vectors.)

Equation (2.7.15) reveals that, if you have $\mathbf{A}^{-1}$ in storage, then you can either make the P corrections in one fell swoop by using (2.7.12) and inverting a $P \times P$ matrix, or else make them by applying (2.7.5) P successive times.

If you don't have storage for $\mathbf{A}^{-1}$, then you *must* use (2.7.12) in the following way: To solve the linear equation

$$\left(\mathbf{A} + \sum_{k=0}^{P-1} \mathbf{u}_k \otimes \mathbf{v}_k\right) \cdot \mathbf{x} = \mathbf{b} \tag{2.7.16}$$

first solve the P auxiliary problems

$$\begin{aligned} \mathbf{A} \cdot \mathbf{z}_0 &= \mathbf{u}_0 \\ \mathbf{A} \cdot \mathbf{z}_1 &= \mathbf{u}_1 \\ &\cdots \\ \mathbf{A} \cdot \mathbf{z}_{P-1} &= \mathbf{u}_{P-1} \end{aligned} \tag{2.7.17}$$

and construct the matrix $\mathbf{Z}$ by columns from the $\mathbf{z}$'s obtained,

$$\mathbf{Z} \equiv \begin{bmatrix} \\ \mathbf{z}_0 \\ \\ \end{bmatrix} \cdots \begin{bmatrix} \\ \mathbf{z}_{P-1} \\ \\ \end{bmatrix} \tag{2.7.18}$$

Next, do the $P \times P$ matrix inversion

$$\mathbf{H} \equiv (\mathbf{1} + \mathbf{V}^T \cdot \mathbf{Z})^{-1} \tag{2.7.19}$$

Finally, solve the one further auxiliary problem

$$\mathbf{A} \cdot \mathbf{y} = \mathbf{b} \tag{2.7.20}$$

In terms of these quantities, the solution is given by

$$\mathbf{x} = \mathbf{y} - \mathbf{Z} \cdot \left[\mathbf{H} \cdot (\mathbf{V}^T \cdot \mathbf{y})\right] \tag{2.7.21}$$

2.7.4 Inversion by Partitioning

Once in a while, you will encounter a matrix (not even necessarily sparse) that can be inverted efficiently by partitioning. Suppose that the $N \times N$ matrix $\mathbf{A}$ is partitioned into

$$\mathbf{A} = \begin{bmatrix} \mathbf{P} & \mathbf{Q} \\ \mathbf{R} & \mathbf{S} \end{bmatrix} \tag{2.7.22}$$

where $\mathbf{P}$ and $\mathbf{S}$ are square matrices of size $p \times p$ and $s \times s$, respectively ($p + s = N$). The matrices $\mathbf{Q}$ and $\mathbf{R}$ are not necessarily square and have sizes $p \times s$ and $s \times p$, respectively.

If the inverse of $\mathbf{A}$ is partitioned in the same manner,

$$\mathbf{A}^{-1} = \begin{bmatrix} \tilde{\mathbf{P}} & \tilde{\mathbf{Q}} \\ \tilde{\mathbf{R}} & \tilde{\mathbf{S}} \end{bmatrix} \tag{2.7.23}$$

then $\tilde{\mathbf{P}}$, $\tilde{\mathbf{Q}}$, $\tilde{\mathbf{R}}$, $\tilde{\mathbf{S}}$, which have the same sizes as $\mathbf{P}$, $\mathbf{Q}$, $\mathbf{R}$, $\mathbf{S}$, respectively, can be found by either the formulas

$$\begin{aligned}
\tilde{\mathbf{P}} &= (\mathbf{P} - \mathbf{Q}\cdot\mathbf{S}^{-1}\cdot\mathbf{R})^{-1} \\
\tilde{\mathbf{Q}} &= -(\mathbf{P} - \mathbf{Q}\cdot\mathbf{S}^{-1}\cdot\mathbf{R})^{-1}\cdot(\mathbf{Q}\cdot\mathbf{S}^{-1}) \\
\tilde{\mathbf{R}} &= -(\mathbf{S}^{-1}\cdot\mathbf{R})\cdot(\mathbf{P} - \mathbf{Q}\cdot\mathbf{S}^{-1}\cdot\mathbf{R})^{-1} \\
\tilde{\mathbf{S}} &= \mathbf{S}^{-1} + (\mathbf{S}^{-1}\cdot\mathbf{R})\cdot(\mathbf{P} - \mathbf{Q}\cdot\mathbf{S}^{-1}\cdot\mathbf{R})^{-1}\cdot(\mathbf{Q}\cdot\mathbf{S}^{-1})
\end{aligned} \tag{2.7.24}$$

or else by the equivalent formulas

$$\begin{aligned}
\tilde{\mathbf{P}} &= \mathbf{P}^{-1} + (\mathbf{P}^{-1}\cdot\mathbf{Q})\cdot(\mathbf{S} - \mathbf{R}\cdot\mathbf{P}^{-1}\cdot\mathbf{Q})^{-1}\cdot(\mathbf{R}\cdot\mathbf{P}^{-1}) \\
\tilde{\mathbf{Q}} &= -(\mathbf{P}^{-1}\cdot\mathbf{Q})\cdot(\mathbf{S} - \mathbf{R}\cdot\mathbf{P}^{-1}\cdot\mathbf{Q})^{-1} \\
\tilde{\mathbf{R}} &= -(\mathbf{S} - \mathbf{R}\cdot\mathbf{P}^{-1}\cdot\mathbf{Q})^{-1}\cdot(\mathbf{R}\cdot\mathbf{P}^{-1}) \\
\tilde{\mathbf{S}} &= (\mathbf{S} - \mathbf{R}\cdot\mathbf{P}^{-1}\cdot\mathbf{Q})^{-1}
\end{aligned} \tag{2.7.25}$$

The parentheses in equations (2.7.24) and (2.7.25) highlight repeated factors that you may wish to compute only once. (Of course, by associativity, you can instead do the matrix multiplications in any order you like.) The choice between using equations (2.7.24) and (2.7.25) depends on whether you want $\tilde{\mathbf{P}}$ or $\tilde{\mathbf{S}}$ to have the simpler formula; or on whether the repeated expression $(\mathbf{S}-\mathbf{R}\cdot\mathbf{P}^{-1}\cdot\mathbf{Q})^{-1}$ is easier to calculate than the expression $(\mathbf{P} - \mathbf{Q}\cdot\mathbf{S}^{-1}\cdot\mathbf{R})^{-1}$; or on the relative sizes of $\mathbf{P}$ and $\mathbf{S}$; or on whether $\mathbf{P}^{-1}$ or $\mathbf{S}^{-1}$ is already known.

Another sometimes useful formula is for the determinant of the partitioned matrix,

$$\det\mathbf{A} = \det\mathbf{P}\,\det(\mathbf{S} - \mathbf{R}\cdot\mathbf{P}^{-1}\cdot\mathbf{Q}) = \det\mathbf{S}\,\det(\mathbf{P} - \mathbf{Q}\cdot\mathbf{S}^{-1}\cdot\mathbf{R}) \tag{2.7.26}$$

2.7.5 Indexed Storage of Sparse Matrices

We have already seen (§2.4) that tri- or band-diagonal matrices can be stored in a compact format that allocates storage only to elements that can be nonzero, plus perhaps a few wasted locations to make the bookkeeping easier. What about more general sparse matrices? When a sparse matrix of dimension $M \times N$ contains only a few times M or N nonzero elements (a typical case), it is surely inefficient — and often physically impossible — to allocate storage for all MN elements. Even if one did allocate such storage, it would be inefficient or prohibitive in machine time to loop over all of it in search of nonzero elements.

Obviously some kind of indexed storage scheme is required, one that stores only nonzero matrix elements, along with sufficient auxiliary information to determine where an element logically belongs and how the various elements can be looped over in common matrix operations. Unfortunately, there is no one standard scheme in general use. Each scheme has its own pluses and minuses, depending on the application.

Before we look at sparse matrices, let's consider the simpler problem of a *sparse vector*. The obvious data structure is a list of the nonzero values and another list of the corresponding locations:

sparse.h

```
struct NRsparseCol
Sparse vector data structure.
{
    Int nrows;                  Number of rows.
    Int nvals;                  Maximum number of nonzeros.
```

```
    VecInt row_ind;                                  Row indices of nonzeros.
    VecDoub val;                                     Array of nonzero values.

    NRsparseCol(Int m,Int nnvals) : nrows(m), nvals(nnvals),
    row_ind(nnvals,0),val(nnvals,0.0) {}             Constructor. Initializes vector to zero.

    NRsparseCol() : nrows(0),nvals(0),row_ind(),val() {}     Default constructor.

    void resize(Int m, Int nnvals) {
        nrows = m;
        nvals = nnvals;
        row_ind.assign(nnvals,0);
        val.assign(nnvals,0.0);
    }

};
```

While we think of this as defining a column vector, you can use exactly the same data structure for a row vector — just mentally interchange the meaning of row and column for the variables. For matrices, however, we have to decide ahead of time whether to use row-oriented or column-oriented storage.

One simple scheme is to use a vector of sparse columns:

```
NRvector<NRsparseCol *> a;
for (i=0;i<n;i++) {
    nvals=...
    a[i]=new NRsparseCol(m,nvals);
}
```

Each column is filled with statements like

```
count=0;
for (j=...) {
    a[i]->row_ind[count]=...
    a[i]->val[count]=...
    count++;
}
```

This data structure is good for an algorithm that primarily works with columns of the matrix, but it is not very efficient when one needs to loop over all elements of the matrix.

A good general storage scheme is the *compressed column storage* format. It is sometimes called the Harwell-Boeing format, after the two large organizations that first systematically provided a standard collection of sparse matrices for research purposes. In this scheme, three vectors are used: `val` for the nonzero values as they are traversed column by column, `row_ind` for the corresponding row indices of each value, and `col_ptr` for the locations in the other two arrays that start a column. In other words, if `val[k]=a[i][j]`, then `row_ind[k]=i`. The first nonzero in column `j` is at `col_ptr[j]`. The last is at `col_ptr[j+1]-1`. Note that `col_ptr[0]` is always 0, and by convention we define `col_ptr[n]` equal to the number of nonzeros. Note also that the dimension of the `col_ptr` array is $N + 1$, not N. The advantage of this scheme is that it requires storage of only about two times the number of nonzero matrix elements. (Other methods can require as much as three or five times.)

As an example, consider the matrix

$$\begin{bmatrix} 3.0 & 0.0 & 1.0 & 2.0 & 0.0 \\ 0.0 & 4.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 7.0 & 5.0 & 9.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 6.0 & 5.0 \end{bmatrix} \tag{2.7.27}$$

In compressed column storage mode, matrix (2.7.27) is represented by two arrays of length 9 and an array of length 6, as follows

index k	0	1	2	3	4	5	6	7	8
val[k]	3.0	4.0	7.0	1.0	5.0	2.0	9.0	6.0	5.0
row_ind[k]	0	1	2	0	2	0	2	4	4

(2.7.28)

index i	0	1	2	3	4	5
col_ptr[i]	0	1	3	5	8	9

Notice that, according to the storage rules, the value of N (namely 5) is the maximum valid index in `col_ptr`. The value of `col_ptr[5]` is 9, the length of the other two arrays. The elements 1.0 and 5.0 in column number 2, for example, are located in positions `col_ptr[2]` $\le k <$ `col_ptr[3]`.

Here is a data structure to handle this storage scheme:

sparse.h
```
struct NRsparseMat
Sparse matrix data structure for compressed column storage.
{
    Int nrows;                          Number of rows.
    Int ncols;                          Number of columns.
    Int nvals;                          Maximum number of nonzeros.
    VecInt col_ptr;                     Pointers to start of columns. Length is ncols+1.
    VecInt row_ind;                     Row indices of nonzeros.
    VecDoub val;                        Array of nonzero values.

    NRsparseMat();                          Default constructor.
    NRsparseMat(Int m,Int n,Int nnvals);    Constructor. Initializes vector to zero.
    VecDoub ax(const VecDoub &x) const;     Multiply A by a vector x[0..ncols-1].
    VecDoub atx(const VecDoub &x) const;    Multiply A^T by a vector x[0..nrows-1].
    NRsparseMat transpose() const;          Form A^T.
};
```

The code for the constructors is standard:

sparse.h
```
NRsparseMat::NRsparseMat() : nrows(0),ncols(0),nvals(0),col_ptr(),
    row_ind(),val() {}
NRsparseMat::NRsparseMat(Int m,Int n,Int nnvals) : nrows(m),ncols(n),
    nvals(nnvals),col_ptr(n+1,0),row_ind(nnvals,0),val(nnvals,0.0) {}
```

The single most important use of a matrix in compressed column storage mode is to multiply a vector to its right. Don't implement this by traversing the rows of $\mathbf{A}$, which is extremely inefficient in this storage mode. Here's the right way to do it:

sparse.h
```
VecDoub NRsparseMat::ax(const VecDoub &x) const {
    VecDoub y(nrows,0.0);
    for (Int j=0;j<ncols;j++) {
        for (Int i=col_ptr[j];i<col_ptr[j+1];i++)
            y[row_ind[i]] += val[i]*x[j];
    }
    return y;
}
```

Some inefficiency occurs because of the indirect addressing. While there are other storage modes that minimize this, they have their own drawbacks.

It is also simple to multiply the *transpose* of a matrix by a vector to its right, since we just traverse the columns directly. (Indirect addressing is still required.) Note that the transpose matrix is not actually constructed.

```
VecDoub NRsparseMat::atx(const VecDoub &x) const {                    sparse.h
    VecDoub y(ncols);
    for (Int i=0;i<ncols;i++) {
        y[i]=0.0;
        for (Int j=col_ptr[i];j<col_ptr[i+1];j++)
            y[i] += val[j]*x[row_ind[j]];
    }
    return y;
}
```

Because the choice of compressed column storage treats rows and columns quite differently, it is rather an involved operation to construct the transpose of a matrix, given the matrix itself in compressed column storage mode. When the operation cannot be avoided, it is

```
NRsparseMat NRsparseMat::transpose() const {                          sparse.h
    Int i,j,k,index,m=nrows,n=ncols;
    NRsparseMat at(n,m,nvals);              Initialized to zero.
    First find the column lengths for A^T, i.e. the row lengths of A.
    VecInt count(m,0);                      Temporary counters for each row of A.
    for (i=0;i<n;i++)
        for (j=col_ptr[i];j<col_ptr[i+1];j++) {
            k=row_ind[j];
            count[k]++;
        }
    for (j=0;j<m;j++)                       Now set at.col_ptr. 0th entry stays 0.
        at.col_ptr[j+1]=at.col_ptr[j]+count[j];
    for(j=0;j<m;j++)                        Reset counters to zero.
        count[j]=0;
    for (i=0;i<n;i++)                       Main loop.
        for (j=col_ptr[i];j<col_ptr[i+1];j++) {
            k=row_ind[j];
            index=at.col_ptr[k]+count[k];   Element's position in column of A^T.
            at.row_ind[index]=i;
            at.val[index]=val[j];
            count[k]++;                     Increment counter for next element in that
        }                                       column.
    return at;
}
```

The only sparse matrix-matrix multiply routine we give is to form the product $\mathbf{ADA}^T$, where $\mathbf{D}$ is a diagonal matrix. This particular product is used to form the so-called normal equations in the interior-point method for linear programming (§10.11). We encapsulate the algorithm in its own structure, `ADAT`:

```
struct ADAT {                                                         sparse.h
    const NRsparseMat &a,&at;               Store references to A and A^T.
    NRsparseMat *adat;                      This will hold ADA^T.

    ADAT(const NRsparseMat &A,const NRsparseMat &AT);
    Allocates compressed column storage for AA^T, where A and A^T are input in compressed
    column format, and fills in values of col_ptr and row_ind. Each column must be in sorted
    order in input matrices. Matrix is output with each column sorted.
    void updateD(const VecDoub &D);
    Computes ADA^T, where D is a diagonal matrix. This function can be called repeatedly
    to update ADA^T for fixed A.
    NRsparseMat &ref();
    Returns reference to adat, which holds ADA^T.
    ~ADAT();
};
```

The algorithm proceeds in two steps. First, the nonzero pattern of $\mathbf{AA}^T$ is found by a call to the constructor. Since $\mathbf{D}$ is diagonal, $\mathbf{AA}^T$ and $\mathbf{ADA}^T$ have the same nonzero structure. Algorithms using `ADAT` will typically have both $\mathbf{A}$ and $\mathbf{A}^T$ available, so we pass them both to the constructor rather than recompute $\mathbf{A}^T$ from $\mathbf{A}$. The constructor allocates storage and assigns values to `col_ptr` and `row_ind`. The structure of `ADAT` is returned with columns in sorted order because routines like the AMD ordering algorithm used in §10.11 require it.

sparse.h

```
ADAT::ADAT(const NRsparseMat &A,const NRsparseMat &AT) : a(A), at(AT) {
    Int h,i,j,k,l,nvals,m=AT.ncols;
    VecInt done(m);
    for (i=0;i<m;i++)                       Initialize to not done.
        done[i]=-1;
    nvals=0;                                First pass determines number of nonzeros.
    for (j=0;j<m;j++) {                     Outer loop over columns of A^T in AA^T.
        for (i=AT.col_ptr[j];i<AT.col_ptr[j+1];i++) {
            k=AT.row_ind[i];                A^T_{kj} ≠ 0. Find column k in first matrix, A.
            for (l=A.col_ptr[k];l<A.col_ptr[k+1];l++) {
                h=A.row_ind[l];             A_{hl} ≠ 0.
                if (done[h] != j) {         Test if contribution already included.
                    done[h]=j;
                    nvals++;
                }
            }
        }
    }
    adat = new NRsparseMat(m,m,nvals);      Allocate storage for ADAT.
    for (i=0;i<m;i++)                       Re-initialize.
        done[i]=-1;
    nvals=0;
    Second pass: Determine columns of adat. Code is identical to first pass except adat->col_ptr
    and adat->row_ind get assigned at appropriate places.
    for (j=0;j<m;j++) {
        adat->col_ptr[j]=nvals;
        for (i=AT.col_ptr[j];i<AT.col_ptr[j+1];i++) {
            k=AT.row_ind[i];
            for (l=A.col_ptr[k];l<A.col_ptr[k+1];l++) {
                h=A.row_ind[l];
                if (done[h] != j) {
                    done[h]=j;
                    adat->row_ind[nvals]=h;
                    nvals++;
                }
            }
        }
    }
    adat->col_ptr[m]=nvals;                 Set last value.
    for (j=0;j<m;j++) {                     Sort columns
        i=adat->col_ptr[j];
        Int size=adat->col_ptr[j+1]-i;
        if (size > 1) {
            VecInt col(size,&adat->row_ind[i]);
            sort(col);
            for (k=0;k<size;k++)
                adat->row_ind[i+k]=col[k];
        }
    }
}
```

The next routine, `updateD`, actually fills in the values in the `val` array. It can be called repeatedly to update $\mathbf{ADA}^T$ for fixed $\mathbf{A}$.

sparse.h
```
void ADAT::updateD(const VecDoub &D) {
    Int h,i,j,k,l,m=a.nrows,n=a.ncols;
    VecDoub temp(n),temp2(m,0.0);
    for (i=0;i<m;i++) {                    Outer loop over columns of A^T.
        for (j=at.col_ptr[i];j< at.col_ptr[i+1];j++) {
            k=at.row_ptr[j];               Scale elements of each column with D and
            temp[k]=at.val[j]*D[k];            store in temp.
        }
        for (j=at.col_ptr[i];j<at.col_ptr[i+1];j++) {        Go down column again.
            k=at.row_ptr[j];
            for (l=a.col_ptr[k];l<a.col_ptr[k+1];l++) {      Go down column k in
                h=a.row_ptr[l];                                  A.
                temp2[h] += temp[k]*a.val[l];        All terms from temp[k] used here.
            }
        }
        for (j=adat->col_ptr[i];j<adat->col_ptr[i+1];j++) {
        Store temp2 in column of answer.
            k=adat->row_ptr[j];
            adat->val[j]=temp2[k];
            temp2[k]=0.0;                  Restore temp2.
        }
    }
}
```

The final two functions are simple. The `ref` routine returns a reference to the matrix $\mathbf{A}\mathbf{D}\mathbf{A}^T$ stored in the structure for other routines that may need to work with it. And the destructor releases the storage.

sparse.h
```
NRsparseMat & ADAT::ref() {
    return *adat;
}

ADAT::~ADAT() {
    delete adat;
}
```

By the way, if you invoke `ADAT` with *different* matrices $\mathbf{A}$ and $\mathbf{B}^T$, everything will work fine as long as $\mathbf{A}$ and $\mathbf{B}$ have the same nonzero pattern.

In *Numerical Recipes* second edition, we gave a related sparse matrix storage mode in which the diagonal of the matrix is stored first, followed by the off-diagonal elements. We now feel that the added complexity of that scheme is not worthwhile for any of the uses in this book. For a discussion of this and other storage schemes, see [7,8]. To see how to work with the diagonal in the compressed column mode, look at the code for `asolve` at the end of this section.

2.7.6 Conjugate Gradient Method for a Sparse System

So-called *conjugate gradient methods* provide a quite general means for solving the $N \times N$ linear system

$$\mathbf{A} \cdot \mathbf{x} = \mathbf{b} \tag{2.7.29}$$

The attractiveness of these methods for large sparse systems is that they reference $\mathbf{A}$ only through its multiplication of a vector, or the multiplication of its transpose and a vector. As we have seen, these operations can be very efficient for a properly stored sparse matrix. You, the "owner" of the matrix $\mathbf{A}$, can be asked to provide functions that perform these sparse matrix multiplications as efficiently as possible. We, the "grand strategists," supply an abstract base class, `Linbcg` below, that contains the method for solving the set of linear equations, (2.7.29), using your functions.

The simplest, "ordinary" conjugate gradient algorithm [9-11] solves (2.7.29) only in the case that $\mathbf{A}$ is symmetric and positive-definite. It is based on the idea of minimizing the function

$$f(\mathbf{x}) = \tfrac{1}{2}\, \mathbf{x} \cdot \mathbf{A} \cdot \mathbf{x} - \mathbf{b} \cdot \mathbf{x} \tag{2.7.30}$$

This function is minimized when its gradient

$$\nabla f = \mathbf{A} \cdot \mathbf{x} - \mathbf{b} \tag{2.7.31}$$

is zero, which is equivalent to (2.7.29). The minimization is carried out by generating a succession of search directions $\mathbf{p}_k$ and improved minimizers $\mathbf{x}_k$. At each stage a quantity α_k is found that minimizes $f(\mathbf{x}_k + \alpha_k \mathbf{p}_k)$, and $\mathbf{x}_{k+1}$ is set equal to the new point $\mathbf{x}_k + \alpha_k \mathbf{p}_k$. The $\mathbf{p}_k$ and $\mathbf{x}_k$ are built up in such a way that $\mathbf{x}_{k+1}$ is also the minimizer of f over the whole vector space of directions already taken, $\{\mathbf{p}_0, \mathbf{p}_1, \ldots, \mathbf{p}_{k-1}\}$. After N iterations you arrive at the minimizer over the entire vector space, i.e., the solution to (2.7.29).

Later, in §10.8, we will generalize this "ordinary" conjugate gradient algorithm to the minimization of arbitrary nonlinear functions. Here, where our interest is in solving linear, but not necessarily positive-definite or symmetric, equations, a different generalization is important, the *biconjugate gradient method*. This method does not, in general, have a simple connection with function minimization. It constructs four sequences of vectors, $\mathbf{r}_k$, $\bar{\mathbf{r}}_k$, $\mathbf{p}_k$, $\bar{\mathbf{p}}_k$, $k = 0, 1, \ldots$. You supply the initial vectors $\mathbf{r}_0$ and $\bar{\mathbf{r}}_0$, and set $\mathbf{p}_0 = \mathbf{r}_0$, $\bar{\mathbf{p}}_0 = \bar{\mathbf{r}}_0$. Then you carry out the following recurrence:

$$\begin{aligned}
\alpha_k &= \frac{\bar{\mathbf{r}}_k \cdot \mathbf{r}_k}{\bar{\mathbf{p}}_k \cdot \mathbf{A} \cdot \mathbf{p}_k} \\
\mathbf{r}_{k+1} &= \mathbf{r}_k - \alpha_k \mathbf{A} \cdot \mathbf{p}_k \\
\bar{\mathbf{r}}_{k+1} &= \bar{\mathbf{r}}_k - \alpha_k \mathbf{A}^T \cdot \bar{\mathbf{p}}_k \\
\beta_k &= \frac{\bar{\mathbf{r}}_{k+1} \cdot \mathbf{r}_{k+1}}{\bar{\mathbf{r}}_k \cdot \mathbf{r}_k} \\
\mathbf{p}_{k+1} &= \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k \\
\bar{\mathbf{p}}_{k+1} &= \bar{\mathbf{r}}_{k+1} + \beta_k \bar{\mathbf{p}}_k
\end{aligned} \tag{2.7.32}$$

This sequence of vectors satisfies the *biorthogonality* condition

$$\bar{\mathbf{r}}_i \cdot \mathbf{r}_j = \mathbf{r}_i \cdot \bar{\mathbf{r}}_j = 0, \qquad j < i \tag{2.7.33}$$

and the *biconjugacy* condition

$$\bar{\mathbf{p}}_i \cdot \mathbf{A} \cdot \mathbf{p}_j = \mathbf{p}_i \cdot \mathbf{A}^T \cdot \bar{\mathbf{p}}_j = 0, \qquad j < i \tag{2.7.34}$$

There is also a mutual orthogonality,

$$\bar{\mathbf{r}}_i \cdot \mathbf{p}_j = \mathbf{r}_i \cdot \bar{\mathbf{p}}_j = 0, \qquad j < i \tag{2.7.35}$$

The proof of these properties proceeds by straightforward induction [12]. As long as the recurrence does not break down earlier because one of the denominators is zero, it must terminate after $m \le N$ steps with $\mathbf{r}_m = \bar{\mathbf{r}}_m = 0$. This is basically because after at most N steps you run out of new orthogonal directions to the vectors you've already constructed.

To use the algorithm to solve the system (2.7.29), make an initial guess $\mathbf{x}_0$ for the solution. Choose $\mathbf{r}_0$ to be the *residual*

$$\mathbf{r}_0 = \mathbf{b} - \mathbf{A} \cdot \mathbf{x}_0 \tag{2.7.36}$$

and choose $\bar{\mathbf{r}}_0 = \mathbf{r}_0$. Then form the sequence of improved estimates

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k \tag{2.7.37}$$

while carrying out the recurrence (2.7.32). Equation (2.7.37) guarantees that $\mathbf{r}_{k+1}$ from the recurrence is in fact the residual $\mathbf{b} - \mathbf{A} \cdot \mathbf{x}_{k+1}$ corresponding to $\mathbf{x}_{k+1}$. Since $\mathbf{r}_m = 0$, $\mathbf{x}_m$ is the solution to equation (2.7.29).

While there is no guarantee that this whole procedure will not break down or become unstable for general $\mathbf{A}$, in practice this is rare. More importantly, the exact termination in at most N iterations occurs only with exact arithmetic. Roundoff error means that you should

regard the process as a genuinely iterative procedure, to be halted when some appropriate error criterion is met.

The ordinary conjugate gradient algorithm is the special case of the biconjugate gradient algorithm when $\mathbf{A}$ is symmetric, and we choose $\bar{\mathbf{r}}_0 = \mathbf{r}_0$. Then $\bar{\mathbf{r}}_k = \mathbf{r}_k$ and $\bar{\mathbf{p}}_k = \mathbf{p}_k$ for all k; you can omit computing them and halve the work of the algorithm. This conjugate gradient version has the interpretation of minimizing equation (2.7.30). If $\mathbf{A}$ is positive-definite as well as symmetric, the algorithm cannot break down (in theory!). The `solve` routine `Linbcg` below indeed reduces to the ordinary conjugate gradient method if you input a symmetric $\mathbf{A}$, but it does all the redundant computations.

Another variant of the general algorithm corresponds to a symmetric but non-positive-definite $\mathbf{A}$, with the choice $\bar{\mathbf{r}}_0 = \mathbf{A} \cdot \mathbf{r}_0$ instead of $\bar{\mathbf{r}}_0 = \mathbf{r}_0$. In this case $\bar{\mathbf{r}}_k = \mathbf{A} \cdot \mathbf{r}_k$ and $\bar{\mathbf{p}}_k = \mathbf{A} \cdot \mathbf{p}_k$ for all k. This algorithm is thus equivalent to the ordinary conjugate gradient algorithm, but with all dot products $\mathbf{a} \cdot \mathbf{b}$ replaced by $\mathbf{a} \cdot \mathbf{A} \cdot \mathbf{b}$. It is called the *minimum residual* algorithm, because it corresponds to successive minimizations of the function

$$\Phi(\mathbf{x}) = \tfrac{1}{2}\, \mathbf{r} \cdot \mathbf{r} = \tfrac{1}{2}\, |\mathbf{A} \cdot \mathbf{x} - \mathbf{b}|^2 \tag{2.7.38}$$

where the successive iterates $\mathbf{x}_k$ minimize Φ over the same set of search directions $\mathbf{p}_k$ generated in the conjugate gradient method. This algorithm has been generalized in various ways for unsymmetric matrices. The *generalized minimum residual* method (GMRES; see [13,14]) is probably the most robust of these methods.

Note that equation (2.7.38) gives

$$\nabla \Phi(\mathbf{x}) = \mathbf{A}^T \cdot (\mathbf{A} \cdot \mathbf{x} - \mathbf{b}) \tag{2.7.39}$$

For any nonsingular matrix $\mathbf{A}$, $\mathbf{A}^T \cdot \mathbf{A}$ is symmetric and positive-definite. You might therefore be tempted to solve equation (2.7.29) by applying the ordinary conjugate gradient algorithm to the problem

$$(\mathbf{A}^T \cdot \mathbf{A}) \cdot \mathbf{x} = \mathbf{A}^T \cdot \mathbf{b} \tag{2.7.40}$$

Don't! The condition number of the matrix $\mathbf{A}^T \cdot \mathbf{A}$ is the square of the condition number of $\mathbf{A}$ (see §2.6 for definition of condition number). A large condition number both increases the number of iterations required and limits the accuracy to which a solution can be obtained. It is almost always better to apply the biconjugate gradient method to the original matrix $\mathbf{A}$.

So far we have said nothing about the *rate* of convergence of these methods. The ordinary conjugate gradient method works well for matrices that are well-conditioned, i.e., "close" to the identity matrix. This suggests applying these methods to the *preconditioned* form of equation (2.7.29),

$$(\tilde{\mathbf{A}}^{-1} \cdot \mathbf{A}) \cdot \mathbf{x} = \tilde{\mathbf{A}}^{-1} \cdot \mathbf{b} \tag{2.7.41}$$

The idea is that you might already be able to solve your linear system easily for some $\tilde{\mathbf{A}}$ close to $\mathbf{A}$, in which case $\tilde{\mathbf{A}}^{-1} \cdot \mathbf{A} \approx \mathbf{1}$, allowing the algorithm to converge in fewer steps. The matrix $\tilde{\mathbf{A}}$ is called a *preconditioner* [9], and the overall scheme given here is known as the *preconditioned biconjugate gradient method* or *PBCG*. In the code below, the user-supplied routine `atimes` does sparse matrix multiplication by $\mathbf{A}$, while the user-supplied routine `asolve` effects matrix multiplication by the inverse of the preconditioner $\tilde{\mathbf{A}}^{-1}$.

For efficient implementation, the PBCG algorithm introduces an additional set of vectors $\mathbf{z}_k$ and $\bar{\mathbf{z}}_k$ defined by

$$\tilde{\mathbf{A}} \cdot \mathbf{z}_k = \mathbf{r}_k \qquad \text{and} \qquad \tilde{\mathbf{A}}^T \cdot \bar{\mathbf{z}}_k = \bar{\mathbf{r}}_k \tag{2.7.42}$$

and modifies the definitions of α_k, β_k, $\mathbf{p}_k$, and $\bar{\mathbf{p}}_k$ in equation (2.7.32):

$$\begin{aligned} \alpha_k &= \frac{\bar{\mathbf{r}}_k \cdot \mathbf{z}_k}{\bar{\mathbf{p}}_k \cdot \mathbf{A} \cdot \mathbf{p}_k} \\ \beta_k &= \frac{\bar{\mathbf{r}}_{k+1} \cdot \mathbf{z}_{k+1}}{\bar{\mathbf{r}}_k \cdot \mathbf{z}_k} \\ \mathbf{p}_{k+1} &= \mathbf{z}_{k+1} + \beta_k \mathbf{p}_k \\ \bar{\mathbf{p}}_{k+1} &= \bar{\mathbf{z}}_{k+1} + \beta_k \bar{\mathbf{p}}_k \end{aligned} \tag{2.7.43}$$

To use Linbcg, below, you will need to supply routines that solve the auxiliary linear systems (2.7.42). If you have no idea what to use for the preconditioner $\tilde{\mathbf{A}}$, then use the diagonal part of $\mathbf{A}$, or even the identity matrix, in which case the burden of convergence will be entirely on the biconjugate gradient method itself.

Linbcg's routine solve, below, is based on a program originally written by Anne Greenbaum. (See [11] for a different, less sophisticated, implementation.) There are a few wrinkles you should know about.

What constitutes "good" convergence is rather application-dependent. The routine solve therefore provides for four possibilities, selected by setting the flag itol on input. If itol=1, iteration stops when the quantity $|\mathbf{A}\cdot\mathbf{x}-\mathbf{b}|/|\mathbf{b}|$ is less than the input quantity tol. If itol=2, the required criterion is

$$|\tilde{\mathbf{A}}^{-1}\cdot(\mathbf{A}\cdot\mathbf{x}-\mathbf{b})|/|\tilde{\mathbf{A}}^{-1}\cdot\mathbf{b}| < \texttt{tol} \tag{2.7.44}$$

If itol=3, the routine uses its own estimate of the error in $\mathbf{x}$ and requires its magnitude, divided by the magnitude of $\mathbf{x}$, to be less than tol. The setting itol=4 is the same as itol=3, except that the largest (in absolute value) component of the error and largest component of $\mathbf{x}$ are used instead of the vector magnitude (that is, the L_∞ norm instead of the L_2 norm). You may need to experiment to find which of these convergence criteria is best for your problem.

On output, err is the tolerance actually achieved. If the returned count iter does not indicate that the maximum number of allowed iterations itmax was exceeded, then err should be less than tol. If you want to do further iterations, leave all returned quantities as they are and call the routine again. The routine loses its memory of the spanned conjugate gradient subspace between calls, however, so you should not force it to return more often than about every N iterations.

linbcg.h

```
struct Linbcg {
```

Abstract base class for solving sparse linear equations by the preconditioned biconjugate gradient method. To use, declare a derived class in which the methods atimes and asolve are defined for your problem, along with any data that they need. Then call the solve method.

```
    virtual void asolve(VecDoub_I &b, VecDoub_O &x, const Int itrnsp) = 0;
    virtual void atimes(VecDoub_I &x, VecDoub_O &r, const Int itrnsp) = 0;
    void solve(VecDoub_I &b, VecDoub_IO &x, const Int itol, const Doub tol,
        const Int itmax, Int &iter, Doub &err);
    Doub snrm(VecDoub_I &sx, const Int itol);          Utility used by solve.
};

void Linbcg::solve(VecDoub_I &b, VecDoub_IO &x, const Int itol, const Doub tol,
    const Int itmax, Int &iter, Doub &err)
```

Solves $\mathbf{A}\cdot\mathbf{x}=\mathbf{b}$ for x[0..n-1], given b[0..n-1], by the iterative biconjugate gradient method. On input x[0..n-1] should be set to an initial guess of the solution (or all zeros); itol is 1,2,3, or 4, specifying which convergence test is applied (see text); itmax is the maximum number of allowed iterations; and tol is the desired convergence tolerance. On output, x[0..n-1] is reset to the improved solution, iter is the number of iterations actually taken, and err is the estimated error. The matrix $\mathbf{A}$ is referenced only through the user-supplied routines atimes, which computes the product of either $\mathbf{A}$ or its transpose on a vector, and asolve, which solves $\tilde{\mathbf{A}}\cdot\mathbf{x}=\mathbf{b}$ or $\tilde{\mathbf{A}}^T\cdot\mathbf{x}=\mathbf{b}$ for some preconditioner matrix $\tilde{\mathbf{A}}$ (possibly the trivial diagonal part of $\mathbf{A}$). This routine can be called repeatedly, with itmax$\lesssim$n, to monitor how err decreases; or it can be called once with a sufficiently large value of itmax so that convergence to tol is achieved.

```
{
    Doub ak,akden,bk,bkden=1.0,bknum,bnrm,dxnrm,xnrm,zm1nrm,znrm;
    const Doub EPS=1.0e-14;
    Int j,n=b.size();
    VecDoub p(n),pp(n),r(n),rr(n),z(n),zz(n);
    iter=0;                                  Calculate initial residual.
    atimes(x,r,0);                           Input to atimes is x[0..n-1], output is r[0..n-1];
    for (j=0;j<n;j++) {                          the final 0 indicates that the matrix (not its
        r[j]=b[j]-r[j];                          transpose) is to be used.
        rr[j]=r[j];
    }
```

```
//atimes(r,rr,0);
if (itol == 1) {
    bnrm=snrm(b,itol);
    asolve(r,z,0);
}
else if (itol == 2) {
    asolve(b,z,0);
    bnrm=snrm(z,itol);
    asolve(r,z,0);
}
else if (itol == 3 || itol == 4) {
    asolve(b,z,0);
    bnrm=snrm(z,itol);
    asolve(r,z,0);
    znrm=snrm(z,itol);
} else throw("illegal itol in linbcg");
while (iter < itmax) {
    ++iter;
    asolve(rr,zz,1);
    for (bknum=0.0,j=0;j<n;j++) bknum += z[j]*rr[j];
    Calculate coefficient bk and direction vectors p and pp.
    if (iter == 1) {
        for (j=0;j<n;j++) {
            p[j]=z[j];
            pp[j]=zz[j];
        }
    } else {
        bk=bknum/bkden;
        for (j=0;j<n;j++) {
            p[j]=bk*p[j]+z[j];
            pp[j]=bk*pp[j]+zz[j];
        }
    }
    bkden=bknum;
    atimes(p,z,0);
    for (akden=0.0,j=0;j<n;j++) akden += z[j]*pp[j];
    ak=bknum/akden;
    atimes(pp,zz,1);
    for (j=0;j<n;j++) {
        x[j] += ak*p[j];
        r[j] -= ak*z[j];
        rr[j] -= ak*zz[j];
    }
    asolve(r,z,0);
    if (itol == 1)
        err=snrm(r,itol)/bnrm;
    else if (itol == 2)
        err=snrm(z,itol)/bnrm;
    else if (itol == 3 || itol == 4) {
        zm1nrm=znrm;
        znrm=snrm(z,itol);
        if (abs(zm1nrm-znrm) > EPS*znrm) {
            dxnrm=abs(ak)*snrm(p,itol);
            err=znrm/abs(zm1nrm-znrm)*dxnrm;
        } else {
            err=znrm/bnrm;
            continue;
        }
        xnrm=snrm(x,itol);
        if (err <= 0.5*xnrm) err /= xnrm;
        else {
            err=znrm/bnrm;
            continue;
        }
```

Uncomment this line to get the "minimum residual" variant of the algorithm.

Input to asolve is r[0..n-1], output is z[0..n-1]; the final 0 indicates that the matrix $\widetilde{\mathbf{A}}$ (not its transpose) is to be used.

Main loop.

Final 1 indicates use of transpose matrix $\widetilde{\mathbf{A}}^T$.

Calculate coefficient ak, new iterate x, and new residuals r and rr.

Solve $\widetilde{\mathbf{A}} \cdot \mathbf{z} = \mathbf{r}$ and check stopping criterion.

Error may not be accurate, so loop again.

Error may not be accurate, so loop again.

```
        }
        if (err <= tol) break;
    }
}
```

The `solve` routine uses this short utility for computing vector norms:

linbcg.h

```
Doub Linbcg::snrm(VecDoub_I &sx, const Int itol)
```

Compute one of two norms for a vector `sx[0..n-1]`, as signaled by `itol`. Used by `solve`.

```
{
    Int i,isamax,n=sx.size();
    Doub ans;
    if (itol <= 3) {
        ans = 0.0;
        for (i=0;i<n;i++) ans += SQR(sx[i]);        Vector magnitude norm.
        return sqrt(ans);
    } else {
        isamax=0;
        for (i=0;i<n;i++) {                         Largest component norm.
            if (abs(sx[i]) > abs(sx[isamax])) isamax=i;
        }
        return abs(sx[isamax]);
    }
}
```

Here is an example of a derived class that solves $\mathbf{A}\cdot\mathbf{x} = \mathbf{b}$ for a matrix $\mathbf{A}$ in `NRsparseMat`'s compressed column sparse format. A naive diagonal preconditioner is used.

asolve.h

```
struct NRsparseLinbcg : Linbcg {
    NRsparseMat &mat;
    Int n;
    NRsparseLinbcg(NRsparseMat &matrix) : mat(matrix), n(mat.nrows) {}
```

The constructor just binds a reference to your sparse matrix, making it available to `asolve` and `atimes`. To solve for a right-hand side, you call this object's `solve` method, as defined in the base class.

```
    void atimes(VecDoub_I &x, VecDoub_O &r, const Int itrnsp) {
        if (itrnsp) r=mat.atx(x);
        else r=mat.ax(x);
    }
    void asolve(VecDoub_I &b, VecDoub_O &x, const Int itrnsp) {
        Int i,j;
        Doub diag;
        for (i=0;i<n;i++) {
            diag=0.0;
            for (j=mat.col_ptr[i];j<mat.col_ptr[i+1];j++)
                if (mat.row_ind[j] == i) {
                    diag=mat.val[j];
                    break;
                }
            x[i]=(diag != 0.0 ? b[i]/diag : b[i]);
```

The matrix $\tilde{\mathbf{A}}$ is the diagonal part of $\mathbf{A}$. Since the transpose matrix has the same diagonal, the flag `itrnsp` is not used in this example.

```
        }
    }
};
```

For another example of using a class derived from `Linbcg` to solve a sparse matrix problem, see §3.8.

CITED REFERENCES AND FURTHER READING:

Tewarson, R.P. 1973, *Sparse Matrices* (New York: Academic Press).[1]

Jacobs, D.A.H. (ed.) 1977, *The State of the Art in Numerical Analysis* (London: Academic Press), Chapter I.3 (by J.K. Reid).[2]

George, A., and Liu, J.W.H. 1981, *Computer Solution of Large Sparse Positive Definite Systems* (Englewood Cliffs, NJ: Prentice-Hall).[3]

NAG Fortran Library (Oxford, UK: Numerical Algorithms Group), see 2007+, `http://www.nag.co.uk`.[4]

IMSL Math/Library Users Manual (Houston: IMSL Inc.), see 2007+, `http://www.vni.com/products/imsl`.[5]

Eisenstat, S.C., Gursky, M.C., Schultz, M.H., and Sherman, A.H. 1977, *Yale Sparse Matrix Package*, Technical Reports 112 and 114 (Yale University Department of Computer Science).[6]

Bai, Z., Demmel, J., Dongarra, J. Ruhe, A., and van der Vorst, H. (eds.) 2000, *Templates for the Solution of Algebraic Eigenvalue Problems: A Practical Guide* Ch. 10 (Philadelphia: S.I.A.M.). Online at URL in `http://www.cs.ucdavis.edu/~bai/ET/contents.html`.[7]

SPARSKIT, 2007+, at `http://www-users.cs.umn.edu/~saad/software/SPARSKIT/sparskit.html`.[8]

Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), Chapters 4 and 10, particularly §10.2–§10.3.[9]

Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), Chapter 8.[10]

Baker, L. 1991, *More C Tools for Scientists and Engineers* (New York: McGraw-Hill).[11]

Fletcher, R. 1976, in *Numerical Analysis Dundee 1975*, Lecture Notes in Mathematics, vol. 506, A. Dold and B Eckmann, eds. (Berlin: Springer), pp. 73–89.[12]

PCGPAK User's Guide (New Haven: Scientific Computing Associates, Inc.).[13]

Saad, Y., and Schulz, M. 1986, *SIAM Journal on Scientific and Statistical Computing*, vol. 7, pp. 856–869.[14]

Ueberhuber, C.W. 1997, *Numerical Computation: Methods, Software, and Analysis*, 2 vols. (Berlin: Springer), Chapter 13.

Bunch, J.R., and Rose, D.J. (eds.) 1976, *Sparse Matrix Computations* (New York: Academic Press).

Duff, I.S., and Stewart, G.W. (eds.) 1979, *Sparse Matrix Proceedings 1978* (Philadelphia: S.I.A.M.).

2.8 Vandermonde Matrices and Toeplitz Matrices

In §2.4 the case of a tridiagonal matrix was treated specially, because that particular type of linear system admits a solution in only of order N operations, rather than of order N^3 for the general linear problem. When such particular types exist, it is important to know about them. Your computational savings, should you ever happen to be working on a problem that involves the right kind of particular type, can be enormous.

This section treats two special types of matrices that can be solved in of order N^2 operations, not as good as tridiagonal, but a lot better than the general case. (Other than the operations count, these two types having nothing in common.) Matrices of the first type, termed *Vandermonde matrices*, occur in some problems having to do with the fitting of polynomials, the reconstruction of distributions from their moments, and also other contexts. In this book, for example, a Vandermonde problem crops up in §3.5. Matrices of the second type, termed *Toeplitz matrices*, tend

to occur in problems involving deconvolution and signal processing. In this book, a Toeplitz problem is encountered in §13.7.

These are not the only special types of matrices worth knowing about. The *Hilbert matrices*, whose components are of the form $a_{ij} = 1/(i + j + 1)$, $i, j = 0, \ldots, N-1$, can be inverted by an exact integer algorithm and are very difficult to invert in any other way, since they are notoriously ill-conditioned (see [1] for details). The Sherman-Morrison and Woodbury formulas, discussed in §2.7, can sometimes be used to convert new special forms into old ones. Reference [2] gives some other special forms. We have not found these additional forms to arise as frequently as the two that we now discuss.

2.8.1 Vandermonde Matrices

A Vandermonde matrix of size $N \times N$ is completely determined by N arbitrary numbers $x_0, x_1, \ldots, x_{N-1}$, in terms of which its N^2 components are the integer powers x_i^j, $i, j = 0, \ldots, N-1$. Evidently there are two possible such forms, depending on whether we view the i's as rows and j's as columns, or vice versa. In the former case, we get a linear system of equations that looks like this,

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{N-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{N-1} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_{N-1} & x_{N-1}^2 & \cdots & x_{N-1}^{N-1} \end{bmatrix} \cdot \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_{N-1} \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{N-1} \end{bmatrix} \tag{2.8.1}$$

Performing the matrix multiplication, you will see that this equation solves for the unknown coefficients c_i that fit a polynomial to the N pairs of abscissas and ordinates (x_j, y_j). Precisely this problem will arise in §3.5, and the routine given there will solve (2.8.1) by the method that we are about to describe.

The alternative identification of rows and columns leads to the set of equations

$$\begin{bmatrix} 1 & 1 & \cdots & 1 \\ x_0 & x_1 & \cdots & x_{N-1} \\ x_0^2 & x_1^2 & \cdots & x_{N-1}^2 \\ & & \cdots & \\ x_0^{N-1} & x_1^{N-1} & \cdots & x_{N-1}^{N-1} \end{bmatrix} \cdot \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \cdots \\ w_{N-1} \end{bmatrix} = \begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ \cdots \\ q_{N-1} \end{bmatrix} \tag{2.8.2}$$

Write this out and you will see that it relates to the *problem of moments*: Given the values of N points x_i, find the unknown weights w_i, assigned so as to match the given values q_j of the first N moments. (For more on this problem, consult [3].) The routine given in this section solves (2.8.2).

The method of solution of both (2.8.1) and (2.8.2) is closely related to Lagrange's polynomial interpolation formula, which we will not formally meet until §3.2. Notwithstanding, the following derivation should be comprehensible:

Let $P_j(x)$ be the polynomial of degree $N-1$ defined by

$$P_j(x) = \prod_{\substack{n=0 \\ n \neq j}}^{N-1} \frac{x - x_n}{x_j - x_n} = \sum_{k=0}^{N-1} A_{jk} x^k \tag{2.8.3}$$

Here the meaning of the last equality is to define the components of the matrix A_{ij} as the coefficients that arise when the product is multiplied out and like terms collected.

The polynomial $P_j(x)$ is a function of x generally. But you will notice that it is specifically designed so that it takes on a value of zero at all x_i with $i \neq j$ and has a value of unity at $x = x_j$. In other words,

$$P_j(x_i) = \delta_{ij} = \sum_{k=0}^{N-1} A_{jk} x_i^k \tag{2.8.4}$$

But (2.8.4) says that A_{jk} is exactly the inverse of the matrix of components x_i^k, which appears in (2.8.2), with the subscript as the column index. Therefore the solution of (2.8.2) is just that matrix inverse times the right-hand side,

$$w_j = \sum_{k=0}^{N-1} A_{jk} q_k \tag{2.8.5}$$

As for the transpose problem (2.8.1), we can use the fact that the inverse of the transpose is the transpose of the inverse, so

$$c_j = \sum_{k=0}^{N-1} A_{kj} y_k \tag{2.8.6}$$

The routine in §3.5 implements this.

It remains to find a good way of multiplying out the monomial terms in (2.8.3), in order to get the components of A_{jk}. This is essentially a bookkeeping problem, and we will let you read the routine itself to see how it can be solved. One trick is to define a master $P(x)$ by

$$P(x) \equiv \prod_{n=0}^{N-1} (x - x_n) \tag{2.8.7}$$

work out its coefficients, and then obtain the numerators and denominators of the specific P_j's via synthetic division by the one supernumerary term. (See §5.1 for more on synthetic division.) Since each such division is only a process of order N, the total procedure is of order N^2.

You should be warned that Vandermonde systems are notoriously ill-conditioned, by their very nature. (As an aside anticipating §5.8, the reason is the same as that which makes Chebyshev fitting so impressively accurate: There exist high-order polynomials that are very good uniform fits to zero. Hence roundoff error can introduce rather substantial coefficients of the leading terms of these polynomials.) It is a good idea always to compute Vandermonde problems in double precision or higher.

The routine for (2.8.2) that follows is due to G.B. Rybicki.

```
void vander(VecDoub_I &x, VecDoub_O &w, VecDoub_I &q)                    vander.h
Solves the Vandermonde linear system sum_{i=0}^{N-1} x_i^k w_i = q_k (k = 0,...,N-1). Input consists
of the vectors x[0..n-1] and q[0..n-1]; the vector w[0..n-1] is output.
{
    Int i,j,k,n=q.size();
    Doub b,s,t,xx;
    VecDoub c(n);
    if (n == 1) w[0]=q[0];
    else {
        for (i=0;i<n;i++) c[i]=0.0;          Initialize array.
        c[n-1] = -x[0];                      Coefficients of the master polynomial are found
        for (i=1;i<n;i++) {                      by recursion.
            xx = -x[i];
            for (j=(n-1-i);j<(n-1);j++) c[j] += xx*c[j+1];
            c[n-1] += xx;
        }
        for (i=0;i<n;i++) {                  Each subfactor in turn
```

```
            xx=x[i];
            t=b=1.0;
            s=q[n-1];
            for (k=n-1;k>0;k--) {          is synthetically divided,
                b=c[k]+xx*b;
                s += q[k-1]*b;             matrix-multiplied by the right-hand side,
                t=xx*t+b;
            }
            w[i]=s/t;                      and supplied with a denominator.
        }
    }
}
```

2.8.2 Toeplitz Matrices

An $N \times N$ Toeplitz matrix is specified by giving $2N-1$ numbers R_k, where the index k ranges over $k = -N+1, \ldots, -1, 0, 1, \ldots, N-1$. Those numbers are then emplaced as matrix elements constant along the (upper-left to lower-right) diagonals of the matrix:

$$\begin{bmatrix} R_0 & R_{-1} & R_{-2} & \cdots & R_{-(N-2)} & R_{-(N-1)} \\ R_1 & R_0 & R_{-1} & \cdots & R_{-(N-3)} & R_{-(N-2)} \\ R_2 & R_1 & R_0 & \cdots & R_{-(N-4)} & R_{-(N-3)} \\ \cdots & & & \cdots & & \\ R_{N-2} & R_{N-3} & R_{N-4} & \cdots & R_0 & R_{-1} \\ R_{N-1} & R_{N-2} & R_{N-3} & \cdots & R_1 & R_0 \end{bmatrix} \tag{2.8.8}$$

The linear Toeplitz problem can thus be written as

$$\sum_{j=0}^{N-1} R_{i-j} x_j = y_i \qquad (i = 0, \ldots, N-1) \tag{2.8.9}$$

where the x_j's, $j = 0, \ldots, N-1$, are the unknowns to be solved for.

The Toeplitz matrix is symmetric if $R_k = R_{-k}$ for all k. Levinson [4] developed an algorithm for fast solution of the symmetric Toeplitz problem, by a *bordering method*, that is, a recursive procedure that solves the $(M+1)$-dimensional Toeplitz problem

$$\sum_{j=0}^{M} R_{i-j} x_j^{(M)} = y_i \qquad (i = 0, \ldots, M) \tag{2.8.10}$$

in turn for $M = 0, 1, \ldots$ until $M = N-1$, the desired result, is finally reached. The vector $x_j^{(M)}$ is the result at the Mth stage and becomes the desired answer only when $N-1$ is reached.

Levinson's method is well documented in standard texts (e.g., [5]). The useful fact that the method generalizes to the *nonsymmetric* case seems to be less well known. At some risk of excessive detail, we therefore give a derivation here, due to G.B. Rybicki.

In following a recursion from step M to step $M+1$ we find that our developing solution $x^{(M)}$ changes in this way:

$$\sum_{j=0}^{M} R_{i-j} x_j^{(M)} = y_i \qquad i = 0, \ldots, M \tag{2.8.11}$$

becomes

$$\sum_{j=0}^{M} R_{i-j} x_j^{(M+1)} + R_{i-(M+1)} x_{M+1}^{(M+1)} = y_i \qquad i = 0, \ldots, M+1 \tag{2.8.12}$$

By eliminating y_i we find

$$\sum_{j=0}^{M} R_{i-j}\left(\frac{x_j^{(M)} - x_j^{(M+1)}}{x_{M+1}^{(M+1)}}\right) = R_{i-(M+1)} \qquad i = 0, \ldots, M \tag{2.8.13}$$

or by letting $i \to M - i$ and $j \to M - j$,

$$\sum_{j=0}^{M} R_{j-i} G_j^{(M)} = R_{-(i+1)} \tag{2.8.14}$$

where

$$G_j^{(M)} \equiv \frac{x_{M-j}^{(M)} - x_{M-j}^{(M+1)}}{x_{M+1}^{(M+1)}} \tag{2.8.15}$$

To put this another way,

$$x_{M-j}^{(M+1)} = x_{M-j}^{(M)} - x_{M+1}^{(M+1)} G_j^{(M)} \qquad j = 0, \ldots, M \tag{2.8.16}$$

Thus, if we can use recursion to find the order M quantities $x^{(M)}$ and $G^{(M)}$ *and* the single order $M + 1$ quantity $x_{M+1}^{(M+1)}$, then all of the other $x_j^{(M+1)}$'s will follow. Fortunately, the quantity $x_{M+1}^{(M+1)}$ follows from equation (2.8.12) with $i = M + 1$,

$$\sum_{j=0}^{M} R_{M+1-j} x_j^{(M+1)} + R_0 x_{M+1}^{(M+1)} = y_{M+1} \tag{2.8.17}$$

For the unknown order $M + 1$ quantities $x_j^{(M+1)}$ we can substitute the previous order quantities in G since

$$G_{M-j}^{(M)} = \frac{x_j^{(M)} - x_j^{(M+1)}}{x_{M+1}^{(M+1)}} \tag{2.8.18}$$

The result of this operation is

$$x_{M+1}^{(M+1)} = \frac{\sum_{j=0}^{M} R_{M+1-j} x_j^{(M)} - y_{M+1}}{\sum_{j=0}^{M} R_{M+1-j} G_{M-j}^{(M)} - R_0} \tag{2.8.19}$$

The only remaining problem is to develop a recursion relation for G. Before we do that, however, we should point out that there are actually two distinct sets of solutions to the original linear problem for a nonsymmetric matrix, namely right-hand solutions (which we have been discussing) and left-hand solutions z_i. The formalism for the left-hand solutions differs only in that we deal with the equations

$$\sum_{j=0}^{M} R_{j-i} z_j^{(M)} = y_i \qquad i = 0, \ldots, M \tag{2.8.20}$$

Then, the same sequence of operations on this set leads to

$$\sum_{j=0}^{M} R_{i-j} H_j^{(M)} = R_{i+1} \tag{2.8.21}$$

where

$$H_j^{(M)} \equiv \frac{z_{M-j}^{(M)} - z_{M-j}^{(M+1)}}{z_{M+1}^{(M+1)}} \tag{2.8.22}$$

(compare with 2.8.14 – 2.8.15). The reason for mentioning the left-hand solutions now is that, by equation (2.8.21), the H_j's satisfy exactly the same equation as the x_j's except for the substitution $y_i \to R_{i+1}$ on the right-hand side. Therefore we can quickly deduce from equation (2.8.19) that

$$H_{M+1}^{(M+1)} = \frac{\sum_{j=0}^{M} R_{M+1-j} H_j^{(M)} - R_{M+2}}{\sum_{j=0}^{M} R_{M+1-j} G_{M-j}^{(M)} - R_0} \tag{2.8.23}$$

By the same token, G satisfies the same equation as z, except for the substitution $y_i \to R_{-(i+1)}$. This gives

$$G_{M+1}^{(M+1)} = \frac{\sum_{j=0}^{M} R_{j-M-1} G_j^{(M)} - R_{-M-2}}{\sum_{j=0}^{M} R_{j-M-1} H_{M-j}^{(M)} - R_0} \tag{2.8.24}$$

The same "morphism" also turns equation (2.8.16), and its partner for z, into the final equations

$$\begin{aligned} G_j^{(M+1)} &= G_j^{(M)} - G_{M+1}^{(M+1)} H_{M-j}^{(M)} \\ H_j^{(M+1)} &= H_j^{(M)} - H_{M+1}^{(M+1)} G_{M-j}^{(M)} \end{aligned} \tag{2.8.25}$$

Now, starting with the initial values

$$x_0^{(0)} = y_0/R_0 \qquad G_0^{(0)} = R_{-1}/R_0 \qquad H_0^{(0)} = R_1/R_0 \tag{2.8.26}$$

we can recurse away. At each stage M we use equations (2.8.23) and (2.8.24) to find $H_{M+1}^{(M+1)}$, $G_{M+1}^{(M+1)}$, and then equation (2.8.25) to find the other components of $H^{(M+1)}, G^{(M+1)}$. From there the vectors $x^{(M+1)}$ and/or $z^{(M+1)}$ are easily calculated.

The program below does this. It incorporates the second equation in (2.8.25) in the form

$$H_{M-j}^{(M+1)} = H_{M-j}^{(M)} - H_{M+1}^{(M+1)} G_j^{(M)} \tag{2.8.27}$$

so that the computation can be done "in place."

Notice that the above algorithm fails if $R_0 = 0$. In fact, because the bordering method does not allow pivoting, the algorithm will fail if any of the diagonal principal minors of the original Toeplitz matrix vanish. (Compare with discussion of the tridiagonal algorithm in §2.4.) If the algorithm fails, your matrix is not necessarily singular — you might just have to solve your problem by a slower and more general algorithm such as LU decomposition with pivoting.

The routine that implements equations (2.8.23) – (2.8.27) is also due to Rybicki. Note that the routine's `r[n-1+j]` is equal to R_j above, so that subscripts on the `r` array vary from 0 to $2N - 2$.

toeplz.h

```
void toeplz(VecDoub_I &r, VecDoub_O &x, VecDoub_I &y)
```

Solves the Toeplitz system $\sum_{j=0}^{N-1} R_{(N-1+i-j)} x_j = y_i$ $(i = 0, \ldots, N-1)$. The Toeplitz matrix need not be symmetric. `y[0..n-1]` and `r[0..2*n-2]` are input arrays; `x[0..n-1]` is the output array.

```
{
    Int j,k,m,m1,m2,n1,n=y.size();
    Doub pp,pt1,pt2,qq,qt1,qt2,sd,sgd,sgn,shn,sxn;
    n1=n-1;
```

```
    if (r[n1] == 0.0) throw("toeplz-1 singular principal minor");
    x[0]=y[0]/r[n1];                    Initialize for the recursion.
    if (n1 == 0) return;
    VecDoub g(n1),h(n1);
    g[0]=r[n1-1]/r[n1];
    h[0]=r[n1+1]/r[n1];
    for (m=0;m<n;m++) {                 Main loop over the recursion.
        m1=m+1;
        sxn = -y[m1];                   Compute numerator and denominator for x from eq.
        sd = -r[n1];                        (2.8.19),
        for (j=0;j<m+1;j++) {
            sxn += r[n1+m1-j]*x[j];
            sd += r[n1+m1-j]*g[m-j];
        }
        if (sd == 0.0) throw("toeplz-2 singular principal minor");
        x[m1]=sxn/sd;                   whence x.
        for (j=0;j<m+1;j++)             Eq. (2.8.16).
            x[j] -= x[m1]*g[m-j];
        if (m1 == n1) return;
        sgn = -r[n1-m1-1];              Compute numerator and denominator for G and H,
        shn = -r[n1+m1+1];                  eqs. (2.8.24) and (2.8.23),
        sgd = -r[n1];
        for (j=0;j<m+1;j++) {
            sgn += r[n1+j-m1]*g[j];
            shn += r[n1+m1-j]*h[j];
            sgd += r[n1+j-m1]*h[m-j];
        }
        if (sgd == 0.0) throw("toeplz-3 singular principal minor");
        g[m1]=sgn/sgd;                  whence G and H.
        h[m1]=shn/sd;
        k=m;
        m2=(m+2) >> 1;
        pp=g[m1];
        qq=h[m1];
        for (j=0;j<m2;j++) {
            pt1=g[j];
            pt2=g[k];
            qt1=h[j];
            qt2=h[k];
            g[j]=pt1-pp*qt2;
            g[k]=pt2-pp*qt1;
            h[j]=qt1-qq*pt2;
            h[k--]=qt2-qq*pt1;
        }
    }                                   Back for another recurrence.
    throw("toeplz - should not arrive here!");
}
```

If you are in the business of solving *very* large Toeplitz systems, you should find out about so-called "new, fast" algorithms, which require only on the order of $N(\log N)^2$ operations, compared to N^2 for Levinson's method. These methods are too complicated to include here. Papers by Bunch [6] and de Hoog [7] will give entry to the literature.

CITED REFERENCES AND FURTHER READING:

Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), Chapter 5 [also treats some other special forms].

Forsythe, G.E., and Moler, C.B. 1967, *Computer Solution of Linear Algebraic Systems* (Englewood Cliffs, NJ: Prentice-Hall), §19.[1]

Westlake, J.R. 1968, *A Handbook of Numerical Matrix Inversion and Solution of Linear Equations* (New York: Wiley).[2]

von Mises, R. 1964, *Mathematical Theory of Probability and Statistics* (New York: Academic Press), pp. 394ff.[3]

Levinson, N., Appendix B of N. Wiener, 1949, *Extrapolation, Interpolation and Smoothing of Stationary Time Series* (New York: Wiley).[4]

Robinson, E.A., and Treitel, S. 1980, *Geophysical Signal Analysis* (Englewood Cliffs, NJ: Prentice-Hall), pp. 163ff.[5]

Bunch, J.R. 1985, "Stability of Methods for Solving Toeplitz Systems of Equations," *SIAM Journal on Scientific and Statistical Computing*, vol. 6, pp. 349–364.[6]

de Hoog, F. 1987, "A New Algorithm for Solving Toeplitz Systems of Equations," *Linear Algebra and Its Applications*, vol. 88/89, pp. 123–138.[7]

2.9 Cholesky Decomposition

If a square matrix $\mathbf{A}$ happens to be symmetric and positive-definite, then it has a special, more efficient, triangular decomposition. *Symmetric* means that $a_{ij} = a_{ji}$ for $i, j = 0, \ldots, N-1$, while *positive-definite* means that

$$\mathbf{v} \cdot \mathbf{A} \cdot \mathbf{v} > 0 \quad \text{for all vectors } \mathbf{v} \tag{2.9.1}$$

(In Chapter 11 we will see that positive-definite has the equivalent interpretation that $\mathbf{A}$ has all positive eigenvalues.) While symmetric, positive-definite matrices are rather special, they occur quite frequently in some applications, so their special factorization, called *Cholesky decomposition*, is good to know about. When you can use it, Cholesky decomposition is about a factor of two faster than alternative methods for solving linear equations.

Instead of seeking arbitrary lower and upper triangular factors $\mathbf{L}$ and $\mathbf{U}$, Cholesky decomposition constructs a lower triangular matrix $\mathbf{L}$ whose transpose $\mathbf{L}^T$ can itself serve as the upper triangular part. In other words we replace equation (2.3.1) by

$$\mathbf{L} \cdot \mathbf{L}^T = \mathbf{A} \tag{2.9.2}$$

This factorization is sometimes referred to as "taking the square root" of the matrix $\mathbf{A}$, though, because of the transpose, it is not literally that. The components of $\mathbf{L}^T$ are of course related to those of $\mathbf{L}$ by

$$L^T_{ij} = L_{ji} \tag{2.9.3}$$

Writing out equation (2.9.2) in components, one readily obtains the analogs of equations (2.3.12) – (2.3.13),

$$L_{ii} = \left(a_{ii} - \sum_{k=0}^{i-1} L_{ik}^2 \right)^{1/2} \tag{2.9.4}$$

and

$$L_{ji} = \frac{1}{L_{ii}} \left(a_{ij} - \sum_{k=0}^{i-1} L_{ik} L_{jk} \right) \qquad j = i+1, i+2, \ldots, N-1 \tag{2.9.5}$$

If you apply equations (2.9.4) and (2.9.5) in the order $i = 0, 1, \ldots, N-1$, you will see that the L's that occur on the right-hand side are already determined by the time they are needed. Also, only components a_{ij} with $j \geq i$ are referenced. (Since $\mathbf{A}$ is symmetric, these have complete information.) If storage is at a premium, it is possible to have the factor $\mathbf{L}$ overwrite the subdiagonal (lower triangular but not including the diagonal) part of $\mathbf{A}$, preserving the input upper triangular values of $\mathbf{A}$; one extra vector of length N is then needed to store the diagonal part of $\mathbf{L}$. The operations count is $N^3/6$ executions of the inner loop (consisting of one multiply and one subtract), with also N square roots. As already mentioned, this is about a factor 2 better than LU decomposition of $\mathbf{A}$ (where its symmetry would be ignored).

You might wonder about pivoting. The pleasant answer is that Cholesky decomposition is extremely stable numerically, without any pivoting at all. Failure of the decomposition simply indicates that the matrix $\mathbf{A}$ (or, with roundoff error, another very nearby matrix) is not positive-definite. In fact, this is an efficient way to test *whether* a symmetric matrix is positive-definite. (In this application, you may want to replace the `throw` in the code below with some less drastic signaling method.)

By now you should be familiar with, if not bored by, our conventions for objects implementing decomposition methods, so we list the object `Cholesky` as a single big mouthful. The methods `elmult` and `elsolve` perform manipulations using the matrix $\mathbf{L}$. The first multiplies $\mathbf{L} \cdot \mathbf{y} = \mathbf{c}$ for a given $\mathbf{y}$, returning $\mathbf{c}$. The second solves this same equation, given $\mathbf{c}$ and returning $\mathbf{y}$. These manipulations are useful in contexts such as multivariate Gaussians (§7.4 and §16.5) and in the analysis of covariance matrices (§15.6).

cholesky.h

```
struct Cholesky{
```

Object for Cholesky decomposition of a matrix $\mathbf{A}$, and related functions.

```
    Int n;
    MatDoub el;                    Stores the decomposition.
    Cholesky(MatDoub_I &a) : n(a.nrows()), el(a) {
```

Constructor. Given a positive-definite symmetric matrix `a[0..n-1][0..n-1]`, construct and store its Cholesky decomposition, $\mathbf{A} = \mathbf{L} \cdot \mathbf{L}^T$.

```
        Int i,j,k;
        VecDoub tmp;
        Doub sum;
        if (el.ncols() != n) throw("need square matrix");
        for (i=0;i<n;i++) {
            for (j=i;j<n;j++) {
                for (sum=el[i][j],k=i-1;k>=0;k--) sum -= el[i][k]*el[j][k];
                if (i == j) {
                    if (sum <= 0.0)          A, with rounding errors, is not positive-definite.
                        throw("Cholesky failed");
                    el[i][i]=sqrt(sum);
                } else el[j][i]=sum/el[i][i];
            }
        }
        for (i=0;i<n;i++) for (j=0;j<i;j++) el[j][i] = 0.;
    }
    void solve(VecDoub_I &b, VecDoub_O &x) {
```

Solve the set of `n` linear equations $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$, where `a` is a positive-definite symmetric matrix whose Cholesky decomposition has been stored. `b[0..n-1]` is input as the right-hand side vector. The solution vector is returned in `x[0..n-1]`.

```
        Int i,k;
        Doub sum;
        if (b.size() != n || x.size() != n) throw("bad lengths in Cholesky");
        for (i=0;i<n;i++) {              Solve L · y = b, storing y in x.
            for (sum=b[i],k=i-1;k>=0;k--) sum -= el[i][k]*x[k];
            x[i]=sum/el[i][i];
        }
        for (i=n-1;i>=0;i--) {           Solve L^T · x = y.
            for (sum=x[i],k=i+1;k<n;k++) sum -= el[k][i]*x[k];
            x[i]=sum/el[i][i];
        }
    }
    void elmult(VecDoub_I &y, VecDoub_O &b) {
```

Multiply $\mathbf{L} \cdot \mathbf{y} = \mathbf{b}$, where $\mathbf{L}$ is the lower triangular matrix in the stored Cholesky decomposition. `y[0..n-1]` is input. The result is returned in `b[0..n-1]`.

```
        Int i,j;
        if (b.size() != n || y.size() != n) throw("bad lengths");
        for (i=0;i<n;i++) {
            b[i] = 0.;
            for (j=0;j<=i;j++) b[i] += el[i][j]*y[j];
        }
    }
```

```
    void elsolve(VecDoub_I &b, VecDoub_O &y) {
```
Solve $\mathbf{L} \cdot \mathbf{y} = \mathbf{b}$, where $\mathbf{L}$ is the lower triangular matrix in the stored Cholesky decomposition. `b[0..n-1]` is input as the right-hand side vector. The solution vector is returned in `y[0..n-1]`.
```
        Int i,j;
        Doub sum;
        if (b.size() != n || y.size() != n) throw("bad lengths");
        for (i=0;i<n;i++) {
            for (sum=b[i],j=0; j<i; j++) sum -= el[i][j]*y[j];
            y[i] = sum/el[i][i];
        }
    }
    void inverse(MatDoub_O &ainv) {
```
Set `ainv[0..n-1][0..n-1]` to the matrix inverse of $\mathbf{A}$, the matrix whose Cholesky decomposition has been stored.
```
        Int i,j,k;
        Doub sum;
        ainv.resize(n,n);
        for (i=0;i<n;i++) for (j=0;j<=i;j++){
            sum = (i==j? 1. : 0.);
            for (k=i-1;k>=j;k--) sum -= el[i][k]*ainv[j][k];
            ainv[j][i]= sum/el[i][i];
        }
        for (i=n-1;i>=0;i--) for (j=0;j<=i;j++){
            sum = (i<j? 0. : ainv[j][i]);
            for (k=i+1;k<n;k++) sum -= el[k][i]*ainv[j][k];
            ainv[i][j] = ainv[j][i] = sum/el[i][i];
        }
    }
    Doub logdet() {
```
Return the logarithm of the determinant of $\mathbf{A}$, the matrix whose Cholesky decomposition has been stored.
```
        Doub sum = 0.;
        for (Int i=0; i<n; i++) sum += log(el[i][i]);
        return 2.*sum;
    }
};
```

CITED REFERENCES AND FURTHER READING:

Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer), Chapter I/1.

Gill, P.E., Murray, W., and Wright, M.H. 1991, *Numerical Linear Algebra and Optimization*, vol. 1 (Redwood City, CA: Addison-Wesley), §4.9.2.

Dahlquist, G., and Bjorck, A. 1974, *Numerical Methods* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2003 (New York: Dover), §5.3.5.

Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), §4.2.

2.10 QR Decomposition

There is another matrix factorization that is sometimes very useful, the so-called QR *decomposition*,

$$\mathbf{A} = \mathbf{Q} \cdot \mathbf{R} \tag{2.10.1}$$

Here $\mathbf{R}$ is upper triangular, while $\mathbf{Q}$ is orthogonal, that is,

$$\mathbf{Q}^T \cdot \mathbf{Q} = \mathbf{1} \tag{2.10.2}$$

where $\mathbf{Q}^T$ is the transpose matrix of $\mathbf{Q}$. Although the decomposition exists for a general rectangular matrix, we shall restrict our treatment to the case when all the matrices are square, with dimensions $N \times N$.

Like the other matrix factorizations we have met (LU, SVD, Cholesky), QR decomposition can be used to solve systems of linear equations. To solve

$$\mathbf{A} \cdot \mathbf{x} = \mathbf{b} \tag{2.10.3}$$

first form $\mathbf{Q}^T \cdot \mathbf{b}$ and then solve

$$\mathbf{R} \cdot \mathbf{x} = \mathbf{Q}^T \cdot \mathbf{b} \tag{2.10.4}$$

by backsubstitution. Since QR decomposition involves about twice as many operations as LU decomposition, it is not used for typical systems of linear equations. However, we will meet special cases where QR is the method of choice.

The standard algorithm for the QR decomposition involves successive Householder transformations (to be discussed later in §11.3). We write a Householder matrix in the form $\mathbf{1} - \mathbf{u} \otimes \mathbf{u}/c$, where $c = \frac{1}{2}\mathbf{u} \cdot \mathbf{u}$. An appropriate Householder matrix applied to a given matrix can zero all elements in a column of the matrix situated below a chosen element. Thus we arrange for the first Householder matrix $\mathbf{Q}_0$ to zero all elements in column 0 of $\mathbf{A}$ below the zeroth element. Similarly, $\mathbf{Q}_1$ zeroes all elements in column 1 below element 1, and so on up to $\mathbf{Q}_{n-2}$. Thus

$$\mathbf{R} = \mathbf{Q}_{n-2} \cdots \mathbf{Q}_0 \cdot \mathbf{A} \tag{2.10.5}$$

Since the Householder matrices are orthogonal,

$$\mathbf{Q} = (\mathbf{Q}_{n-2} \cdots \mathbf{Q}_0)^{-1} = \mathbf{Q}_0 \cdots \mathbf{Q}_{n-2} \tag{2.10.6}$$

In many applications $\mathbf{Q}$ is not needed explicitly, and it is sufficient to store only the factored form (2.10.6). (We do, however, store $\mathbf{Q}$, or rather its transpose, in the code below.) Pivoting is not usually necessary unless the matrix $\mathbf{A}$ is very close to singular. A general QR algorithm for rectangular matrices including pivoting is given in [1]. For square matrices and without pivoting, an implementation is as follows:

qrdcmp.h

```
struct QRdcmp {
```

Object for QR decomposition of a matrix $\mathbf{A}$, and related functions.

```
    Int n;
    MatDoub qt, r;                                  Stored Q^T and R.
    Bool sing;                                      Indicates whether A is singular.
    QRdcmp(MatDoub_I &a);                           Constructor from A.
    void solve(VecDoub_I &b, VecDoub_O &x);         Solve A·x = b for x.
    void qtmult(VecDoub_I &b, VecDoub_O &x);        Multiply Q^T·b = x.
    void rsolve(VecDoub_I &b, VecDoub_O &x);        Solve R·x = b for x.
    void update(VecDoub_I &u, VecDoub_I &v);        See next subsection.
    void rotate(const Int i, const Doub a, const Doub b);      Used by update.
};
```

As usual, the constructor performs the actual decomposition:

qrdcmp.h

```
QRdcmp::QRdcmp(MatDoub_I &a)
    : n(a.nrows()), qt(n,n), r(a), sing(false) {
```

Construct the QR decomposition of `a[0..n-1][0..n-1]`. The upper triangular matrix $\mathbf{R}$ and the transpose of the orthogonal matrix $\mathbf{Q}$ are stored. `sing` is set to `true` if a singularity is encountered during the decomposition, but the decomposition is still completed in this case; otherwise it is set to `false`.

```
    Int i,j,k;
    VecDoub c(n), d(n);
    Doub scale,sigma,sum,tau;
```

```
    for (k=0;k<n-1;k++) {
        scale=0.0;
        for (i=k;i<n;i++) scale=MAX(scale,abs(r[i][k]));
        if (scale == 0.0) {                    Singular case.
            sing=true;
            c[k]=d[k]=0.0;
        } else {                               Form Q_k and Q_k · A.
            for (i=k;i<n;i++) r[i][k] /= scale;
            for (sum=0.0,i=k;i<n;i++) sum += SQR(r[i][k]);
            sigma=SIGN(sqrt(sum),r[k][k]);
            r[k][k] += sigma;
            c[k]=sigma*r[k][k];
            d[k] = -scale*sigma;
            for (j=k+1;j<n;j++) {
                for (sum=0.0,i=k;i<n;i++) sum += r[i][k]*r[i][j];
                tau=sum/c[k];
                for (i=k;i<n;i++) r[i][j] -= tau*r[i][k];
            }
        }
    }
    d[n-1]=r[n-1][n-1];
    if (d[n-1] == 0.0) sing=true;
    for (i=0;i<n;i++) {                        Form Q^T explicitly.
        for (j=0;j<n;j++) qt[i][j]=0.0;
        qt[i][i]=1.0;
    }
    for (k=0;k<n-1;k++) {
        if (c[k] != 0.0) {
            for (j=0;j<n;j++) {
                sum=0.0;
                for (i=k;i<n;i++)
                    sum += r[i][k]*qt[i][j];
                sum /= c[k];
                for (i=k;i<n;i++)
                    qt[i][j] -= sum*r[i][k];
            }
        }
    }
    for (i=0;i<n;i++) {                        Form R explicitly.
        r[i][i]=d[i];
        for (j=0;j<i;j++) r[i][j]=0.0;
    }
}
```

The next set of member functions is used to solve linear systems. In many applications only the part (2.10.4) of the algorithm is needed, so we put in separate routines the multiplication $\mathbf{Q}^T \cdot \mathbf{b}$ and the backsubstitution on $\mathbf{R}$.

qrdcmp.h

```
void QRdcmp::solve(VecDoub_I &b, VecDoub_O &x) {
```

Solve the set of `n` linear equations $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$. `b[0..n-1]` is input as the right-hand side vector, and `x[0..n-1]` is returned as the solution vector.

```
    qtmult(b,x);                   Form Q^T · b.
    rsolve(x,x);                   Solve R · x = Q^T · b.
}

void QRdcmp::qtmult(VecDoub_I &b, VecDoub_O &x) {
```

Multiply $\mathbf{Q}^T \cdot \mathbf{b}$ and put the result in $\mathbf{x}$. Since $\mathbf{Q}$ is orthogonal, this is equivalent to solving $\mathbf{Q} \cdot \mathbf{x} = \mathbf{b}$ for $\mathbf{x}$.

```
    Int i,j;
    Doub sum;
    for (i=0;i<n;i++) {
        sum = 0.;
```

```
        for (j=0;j<n;j++) sum += qt[i][j]*b[j];
        x[i] = sum;
    }
}

void QRdcmp::rsolve(VecDoub_I &b, VecDoub_O &x) {
```

Solve the triangular set of `n` linear equations $\mathbf{R} \cdot \mathbf{x} = \mathbf{b}$. `b[0..n-1]` is input as the right-hand side vector, and `x[0..n-1]` is returned as the solution vector.

```
    Int i,j;
    Doub sum;
    if (sing) throw("attempting solve in a singular QR");
    for (i=n-1;i>=0;i--) {
        sum=b[i];
        for (j=i+1;j<n;j++) sum -= r[i][j]*x[j];
        x[i]=sum/r[i][i];
    }
}
```

See [2] for details on how to use QR decomposition for constructing orthogonal bases, and for solving least-squares problems. (We prefer to use SVD, §2.6, for these purposes, because of its greater diagnostic capability in pathological cases.)

2.10.1 Updating a QR decomposition

Some numerical algorithms involve solving a succession of linear systems each of which differs only slightly from its predecessor. Instead of doing $O(N^3)$ operations each time to solve the equations from scratch, one can often update a matrix factorization in $O(N^2)$ operations and use the new factorization to solve the next set of linear equations. The LU decomposition is complicated to update because of pivoting. However, QR turns out to be quite simple for a very common kind of update,

$$\mathbf{A} \to \mathbf{A} + \mathbf{s} \otimes \mathbf{t} \tag{2.10.7}$$

(compare equation 2.7.1). In practice it is more convenient to work with the equivalent form

$$\mathbf{A} = \mathbf{Q} \cdot \mathbf{R} \quad \to \quad \mathbf{A}' = \mathbf{Q}' \cdot \mathbf{R}' = \mathbf{Q} \cdot (\mathbf{R} + \mathbf{u} \otimes \mathbf{v}) \tag{2.10.8}$$

One can go back and forth between equations (2.10.7) and (2.10.8) using the fact that $\mathbf{Q}$ is orthogonal, giving

$$\mathbf{t} = \mathbf{v} \quad \text{and either} \quad \mathbf{s} = \mathbf{Q} \cdot \mathbf{u} \quad \text{or} \quad \mathbf{u} = \mathbf{Q}^T \cdot \mathbf{s} \tag{2.10.9}$$

The algorithm [2] has two phases. In the first we apply $N-1$ Jacobi rotations (§11.1) to reduce $\mathbf{R} + \mathbf{u} \otimes \mathbf{v}$ to upper Hessenberg form. Another $N-1$ Jacobi rotations transform this upper Hessenberg matrix to the new upper triangular matrix $\mathbf{R}'$. The matrix $\mathbf{Q}'$ is simply the product of $\mathbf{Q}$ with the $2(N-1)$ Jacobi rotations. In applications we usually want $\mathbf{Q}^T$, so the algorithm is arranged to work with this matrix (which is stored in the `QRdcmp` object) instead of with $\mathbf{Q}$.

```
void QRdcmp::update(VecDoub_I &u, VecDoub_I &v) {
```

qrdcmp.h

Starting from the stored QR decomposition $\mathbf{A} = \mathbf{Q} \cdot \mathbf{R}$, update it to be the QR decomposition of the matrix $\mathbf{Q} \cdot (\mathbf{R} + \mathbf{u} \otimes \mathbf{v})$. Input quantities are `u[0..n-1]`, and `v[0..n-1]`.

```
    Int i,k;
    VecDoub w(u);
    for (k=n-1;k>=0;k--)
        if (w[k] != 0.0) break;
    if (k < 0) k=0;
    for (i=k-1;i>=0;i--) {
        rotate(i,w[i],-w[i+1]);
        if (w[i] == 0.0)
            w[i]=abs(w[i+1]);
```

Find largest `k` such that `u[k]` $\neq 0$.

Transform $\mathbf{R} + \mathbf{u} \otimes \mathbf{v}$ to upper Hessenberg.

```
        else if (abs(w[i]) > abs(w[i+1]))
            w[i]=abs(w[i])*sqrt(1.0+SQR(w[i+1]/w[i]));
        else w[i]=abs(w[i+1])*sqrt(1.0+SQR(w[i]/w[i+1]));
    }
    for (i=0;i<n;i++) r[0][i] += w[0]*v[i];
    for (i=0;i<k;i++)                    Transform upper Hessenberg matrix to upper tri-
        rotate(i,r[i][i],-r[i+1][i]);      angular.
    for (i=0;i<n;i++)
        if (r[i][i] == 0.0) sing=true;
}

void QRdcmp::rotate(const Int i, const Doub a, const Doub b)
```

Utility used by `update`. Given matrices `r[0..n-1][0..n-1]` and `qt[0..n-1][0..n-1]`, carry out a Jacobi rotation on rows `i` and `i + 1` of each matrix. `a` and `b` are the parameters of the rotation: $\cos\theta = a/\sqrt{a^2+b^2}$, $\sin\theta = b/\sqrt{a^2+b^2}$.

```
{
    Int j;
    Doub c,fact,s,w,y;
    if (a == 0.0) {                  Avoid unnecessary overflow or underflow.
        c=0.0;
        s=(b >= 0.0 ? 1.0 : -1.0);
    } else if (abs(a) > abs(b)) {
        fact=b/a;
        c=SIGN(1.0/sqrt(1.0+(fact*fact)),a);
        s=fact*c;
    } else {
        fact=a/b;
        s=SIGN(1.0/sqrt(1.0+(fact*fact)),b);
        c=fact*s;
    }
    for (j=i;j<n;j++) {              Premultiply r by Jacobi rotation.
        y=r[i][j];
        w=r[i+1][j];
        r[i][j]=c*y-s*w;
        r[i+1][j]=s*y+c*w;
    }
    for (j=0;j<n;j++) {              Premultiply qt by Jacobi rotation.
        y=qt[i][j];
        w=qt[i+1][j];
        qt[i][j]=c*y-s*w;
        qt[i+1][j]=s*y+c*w;
    }
}
```

We will make use of QR decomposition, and its updating, in §9.7.

CITED REFERENCES AND FURTHER READING:

Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer), Chapter I/8.[1]

Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), §5.2, §5.3, §12.5.[2]

2.11 Is Matrix Inversion an N^3 Process?

We close this chapter with a little entertainment, a bit of algorithmic prestidigitation that probes more deeply into the subject of matrix inversion. We start with a seemingly simple question:

How many individual multiplications does it take to perform the matrix multiplication of two 2×2 matrices,

$$\begin{pmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{pmatrix} \cdot \begin{pmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \end{pmatrix} = \begin{pmatrix} c_{00} & c_{01} \\ c_{10} & c_{11} \end{pmatrix} \tag{2.11.1}$$

Eight, right? Here they are written explicitly:

$$\begin{aligned} c_{00} &= a_{00} \times b_{00} + a_{01} \times b_{10} \\ c_{01} &= a_{00} \times b_{01} + a_{01} \times b_{11} \\ c_{10} &= a_{10} \times b_{00} + a_{11} \times b_{10} \\ c_{11} &= a_{10} \times b_{01} + a_{11} \times b_{11} \end{aligned} \tag{2.11.2}$$

Do you think that one can write formulas for the c's that involve only *seven* multiplications? (Try it yourself, before reading on.)

Such a set of formulas was, in fact, discovered by Strassen [1]. The formulas are

$$\begin{aligned} Q_0 &\equiv (a_{00} + a_{11}) \times (b_{00} + b_{11}) \\ Q_1 &\equiv (a_{10} + a_{11}) \times b_{00} \\ Q_2 &\equiv a_{00} \times (b_{01} - b_{11}) \\ Q_3 &\equiv a_{11} \times (-b_{00} + b_{10}) \\ Q_4 &\equiv (a_{00} + a_{01}) \times b_{11} \\ Q_5 &\equiv (-a_{00} + a_{10}) \times (b_{00} + b_{01}) \\ Q_6 &\equiv (a_{01} - a_{11}) \times (b_{10} + b_{11}) \end{aligned} \tag{2.11.3}$$

in terms of which

$$\begin{aligned} c_{00} &= Q_0 + Q_3 - Q_4 + Q_6 \\ c_{10} &= Q_1 + Q_3 \\ c_{01} &= Q_2 + Q_4 \\ c_{11} &= Q_0 + Q_2 - Q_1 + Q_5 \end{aligned} \tag{2.11.4}$$

What's the use of this? There is one fewer multiplication than in equation (2.11.2), but *many more* additions and subtractions. It is not clear that anything has been gained. But notice that in (2.11.3) the a's and b's are never commuted. Therefore (2.11.3) and (2.11.4) are valid when the a's and b's are themselves matrices. The problem of multiplying two very large matrices (of order $N = 2^m$ for some integer m) can now be broken down recursively by partitioning the matrices into quarters, sixteenths, etc. And note the key point: The savings is not just a factor "7/8"; it is that factor at *each* hierarchical level of the recursion. In total it reduces the process of matrix multiplication to order $N^{\log_2 7}$ instead of N^3.

What about all the extra additions in (2.11.3) – (2.11.4)? Don't they outweigh the advantage of the fewer multiplications? For large N, it turns out that there are six times as many additions as multiplications implied by (2.11.3) – (2.11.4). But, if N is very large, this constant factor is no match for the change in the *exponent* from N^3 to $N^{\log_2 7}$.

With this "fast" matrix multiplication, Strassen also obtained a surprising result for matrix inversion [1]. Suppose that the matrices

$$\begin{pmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{pmatrix} \quad \text{and} \quad \begin{pmatrix} c_{00} & c_{01} \\ c_{10} & c_{11} \end{pmatrix} \tag{2.11.5}$$

are inverses of each other. Then the c's can be obtained from the a's by the following operations (compare equations 2.7.11 and 2.7.25):

$$\begin{aligned} R_0 &= \text{Inverse}(a_{00}) \\ R_1 &= a_{10} \times R_0 \\ R_2 &= R_0 \times a_{01} \\ R_3 &= a_{10} \times R_2 \\ R_4 &= R_3 - a_{11} \\ R_5 &= \text{Inverse}(R_4) \\ c_{01} &= R_2 \times R_5 \\ c_{10} &= R_5 \times R_1 \\ R_6 &= R_2 \times c_{10} \\ c_{00} &= R_0 - R_6 \\ c_{11} &= -R_5 \end{aligned} \tag{2.11.6}$$

In (2.11.6) the "inverse" operator occurs just twice. It is to be interpreted as the reciprocal if the a's and c's are scalars, but as matrix inversion if the a's and c's are themselves submatrices. Imagine doing the inversion of a very large matrix, of order $N = 2^m$, recursively by partitions in half. At each step, halving the order *doubles* the number of inverse operations. But this means that there are only N divisions in all! So divisions don't dominate in the recursive use of (2.11.6). Equation (2.11.6) is dominated, in fact, by its 6 multiplications. Since these can be done by an $N^{\log_2 7}$ algorithm, so can the matrix inversion!

This is fun, but let's look at practicalities: If you estimate how large N has to be before the difference between exponent 3 and exponent $\log_2 7 = 2.807$ is substantial enough to outweigh the bookkeeping overhead, arising from the complicated nature of the recursive Strassen algorithm, you will find that LU decomposition is in no immediate danger of becoming obsolete. However, the fast matrix multiplication routine itself is beginning to appear in libraries like BLAS, where it is typically used for $N \gtrsim 100$.

Strassen's original result for matrix multiplication has been steadily improved. The fastest currently known algorithm [2] has an asymptotic order of $N^{2.376}$, but it is not likely to be practical to implement it.

If you like this kind of fun, then try these: (1) Can you multiply the complex numbers $(a+ib)$ and $(c+id)$ in only *three* real multiplications? [Answer: See §5.5.] (2) Can you evaluate a general fourth-degree polynomial in x for many different values of x with only *three* multiplications per evaluation? [Answer: See §5.1.]

CITED REFERENCES AND FURTHER READING:

Strassen, V. 1969, "Gaussian Elimination Is Not Optimal," *Numerische Mathematik*, vol. 13, pp. 354–356.[1]

Coppersmith, D., and Winograd, S. 1990, "Matrix Multiplications via Arithmetic Progressions," *Journal of Symbolic Computation*, vol. 9, pp. 251–280.[2]

Kronsjö, L. 1987, *Algorithms: Their Complexity and Efficiency*, 2nd ed. (New York: Wiley).

Winograd, S. 1971, "On the Multiplication of 2 by 2 Matrices," *Linear Algebra and Its Applications*, vol. 4, pp. 381–388.

Pan, V. Ya. 1980, "New Fast Algorithms for Matrix Operations," *SIAM Journal on Computing*, vol. 9, pp. 321–342.

Pan, V. 1984, *How to Multiply Matrices Faster*, Lecture Notes in Computer Science, vol. 179 (New York: Springer)

Pan, V. 1984, "How Can We Speed Up Matrix Multiplication?", *SIAM Review*, vol. 26, pp. 393–415.